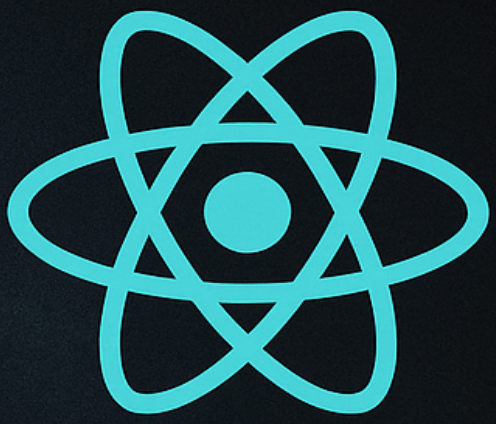


express

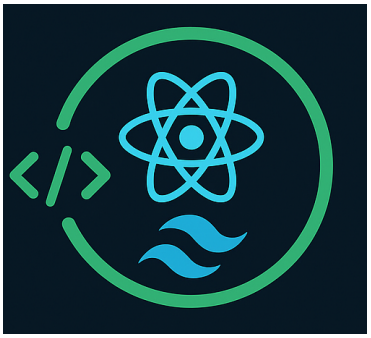
tailwind

XAMPP

Google

SOCKET.IO

reCAPTCHA



WEBologie - Pahae

Guide complet pour le dévelopement Web avec Next.js et des technologies modernes

Auteur : Lamrissi Bahaa-eddine
Téléphone : +212713947508
Email : lam.bahae7@gmail.com
Date de publication : Juin 2025
Localisation : Settat, Maroc
ISBN : 978-3-16-148410-0

Signature
Lamrissi Bahaa-eddine

Tous droits réservés. Toute reproduction est interdite sans l'autorisation de l'auteur.

A propos

Ce livre est une immersion complète dans le monde du développement web moderne avec Next.js comme pilier central. Pensé pour les développeurs familiers avec React.js, il offre une progression fluide vers des concepts avancés en front-end, back-end, et communication en temps réel, tout en intégrant des outils incontournables comme TailwindCSS, GSAP, Socket.IO, WebRTC et bien plus encore.

Table des matières

1 Contexte général de Next.js **8**

I C'est quoi Next.js ? . . . 8

1 Définition et intérêt . . . 8

2 SSR, SSG et ISR . . . 9

3 API REST . . . 9

II Technologies qui vont être expliquées dans ce cours . . . 10

1 Frameworks principaux . . . 10

2 Styling et Cascading . . . 10

3 Authentification et sessions . . . 11

4 Génération de documents . . . 11

5 Communication en temps réel . . . 11

6 SMTP . . . 12

III Préparation de l'environnement et installations . . . 12

1 Installation de Node.js . . . 12

2 Création du premier projet Next.js . . . 12

3 Bases de données . . . 13

2 Guide en React.js pour passer à Next.js **15**

I Rappels en JavaScript . . . 15

1 Destructeurs, Mapping, Filtrage, et Réducteurs des tableaux . . . 15

2 Promesses et fonctions asynchrones . . . 16

3 Modification d'un tableau d'objets avec des fonctions . . . 16

4 Copier un texte dans le clipboard . . . 17

5 Détection de la voix et la transformer en texte . . . 17

II La notion du JSX . . . 18

1 Introduction au JSX . . . 18

2 Différences entre JSX et HTML . . . 18

3 Routes des pages . . . 19

III Le rendring en React . . . 20

1 Structure d'une page . . . 20

2 Components et Props . . . 20

3 Fragments . . . 23

4 Conditional rendring . . . 25

5 Fichier des constantes . . . 26

IV La notion des Hooks . . . 28

1 Variables d'état : useState . . . 28

2 Effets secondaires : useEffect / useLayoutEffect . . . 29

3 Références : useRef . . . 32

4 Valeurs mémorisées : useMemo / useCallback . . . 33
5 Identifiant unique : useId . . . 34
V Styling en React . . . 35
1 Global styles . . . 35
2 Modular styles . . . 36
3 Internal styles . . . 36
4 Inline styles . . . 37
5 JSX styles . . . 37

3 Styling avec TailwindCSS **40**
I Configuration de TailwindCSS . . . 40
II Bases en TailwindCSS . . . 40
1 Couleurs et arrière-plans . . . 40
2 Tailles et positionnements . . . 41
3 Texts et fonts . . . 41
III Mises en page TailwindCSS . . . 41
1 Flexing . . . 41
2 Griding . . . 42
IV Pseudo-classes et group-hover . . . 42
V Animations . . . 42
VI Contrôle de responsivité . . . 43

4 Front-End en Next.js **45**
I Architecture de Next.js . . . 45
II Routage . . . 45
1 Fichier principale . . . 45
2 Le fichier `_document.jsx` dans Next.js . . . 45
3 Hook de routage : useRouter . . . 46
4 Routes . . . 50

5 Backend-End en Next.js **54**
I Conception des APIs . . . 54
1 Emplacement et contexte . . . 54
2 Fonction handler et réponses serveur . . . 54
3 Fetching et récupération de données . . . 55
4 Traitement de la réponse du serveur . . . 60
5 Variables d'environnement (dotenv) . . . 62
II MySQL . . . 64
1 Connexion à la base de données . . . 64
2 Traitement des requêtes . . . 65
III MongoDB . . . 65
1 Connexion à la base de données . . . 65
2 Modèles . . . 66
3 Opérations CRUD et Population . . . 67
IV Gestion des fichiers . . . 72
1 Définition d'un BLOB . . . 72
2 Traitement BDD . . . 73
3 Traitement Front-End . . . 73
4 Traitement Back-End . . . 77

5 BodyParser et gestion de taille limite . . . 77
6 Contrôle du progès d'upload . . . 78
7 Récupération des données . . . 79
8 Téléchargement d'un fichier . . . 81
V Authentification . . . 82
1 Middlewares d'authentification avec JWT . . . 82
2 Login et déclanchement d'une session . . . 84
3 Logout et destruction de la session . . . 85
4 Récupération du compte . . . 85
5 Validation de Captcha . . . 86
6 Récupération et vérification de la session dans les pages . . . 89
VI Fournisseurs d'accès . . . 90
1 Login avec Google . . . 90
2 Login avec Facebook . . . 93
3 Login avec Github . . . 94

6 Communications en temps réel avec Socket.IO 98
I Principe et logique des sockets . . . 98
1 Problème de communication via HTTP . . . 98
2 Solution : TCP/IP . . . 100
II Implémentation d'un système de messagerie . . . 101
1 Configuration d'un serveur WebSocket . . . 101
2 Initialisation des sockets . . . 101
3 Récupération des messages . . . 102
4 Emission d'un message . . . 103
5 Réception d'un message . . . 104
III Implémentation d'un système de notifications . . . 104
1 Emission d'une notification . . . 104
2 Réception d'une notification . . . 105
IV Gestion de salles . . . 107
1 Concept des salles . . . 107
2 Diffusion des messages . . . 107

7 Communications en temps réel avec WebRTC 110
I Pourquoi Next.js ne supporte pas WebRTC ? . . . 110
1 Contraintes techniques . . . 110
2 Conséquences pratiques . . . 110
3 Solution courante . . . 110
II Configuration de WebRTC avec Express.js . . . 110
1 Partie d'initiation Express . . . 111
2 Partie de configuration WebRTC . . . 111
III Création d'une salle et connexion . . . 112

8 APIs et Plugins largrement utilisées 114
I APIs utiles dans le dévelopement web . . . 114
1 Gemini API . . . 114
2 Google-Find-Me API . . . 114
3 Google-Weather API . . . 114
4 Firebase API . . . 114

5 YouTube Data API . . . 114
6 Unsplash Data API . . . 114
II Plugins et bibliothèques utiles dans le dévelopement web . . . 114
1 Facebook-Page Plugin . . . 114
2 React Toastify . . . 114
3 React Lucide Icons . . . 114
4 React Spinners . . . 114
5 AOS (Animate On Scroll) . . . 114
6 React Recharts . . . 114
7 React QRCode . . . 114

9 Hébérgement 116
I Compilation en Next.js . . . 116
1 Scripts de package.json et commandes npm . . . 116
2 Building du projet . . . 116
II GitHub . . . 117
1 Notions de base . . . 117
2 Upload du projet à GitHub . . . 117
3 Problème de remote déjà existant . . . 118
4 Problème d'anciennes sessions . . . 118
5 Modification d'un code déjà uploadé à Github . . . 118
6 Projet en collaboration avec Github . . . 119
III Vercel et son problème . . . 119
1 Déploiement simple sur Vercel . . . 119
2 Limites et problèmes avec Vercel . . . 120
IV Hostinger . . . 120
1 Problèmatique . . . 120
2 Solution VPS . . . 120
V Tâches Cron planifiées . . . 123
1 Définition . . . 123
2 Méthode sans serveur : Cron en ligne . . . 124
3 Étapes sur cron-job.org . . . 124
VI Amélioration de compatibilité . . . 124
1 Transformation en desktop-app avec Electron.js . . . 124
2 Transformation en mobile-app avec Webintoapp . . . 125
VII Docker . . . 125

10 CGI et animations haut niveau 127

Liste des abréviations

API : Application Programming Interface
ARIA : Accessible Rich Internet Applications
BDD : Base de Données
CSS : Cascading Style Sheets
CRUD : Create, Read, Update, Delete
DBMS : Database Management System
GSAP : GreenSock Animation Platform
HTTP : HyperText Transfer Protocol
ID : Identifier
IP : Internet Protocol
JS : JavaScript
JSON : JavaScript Object Notation
JWT : JSON Web Token
MFBA : Modular Folder-Based Architechture
NPM : Node Package Manager
REST : Representational State Transfer
SMTP : Simple Mail Transfer Protocol
URI : Uniform Resource Identifier
URL : Uniform Resource Locator
UI : User Interface
UX : User Experience

Chapitre 1 : Contexte général de Next.js

Chapitre 1

Contexte général de Next.js

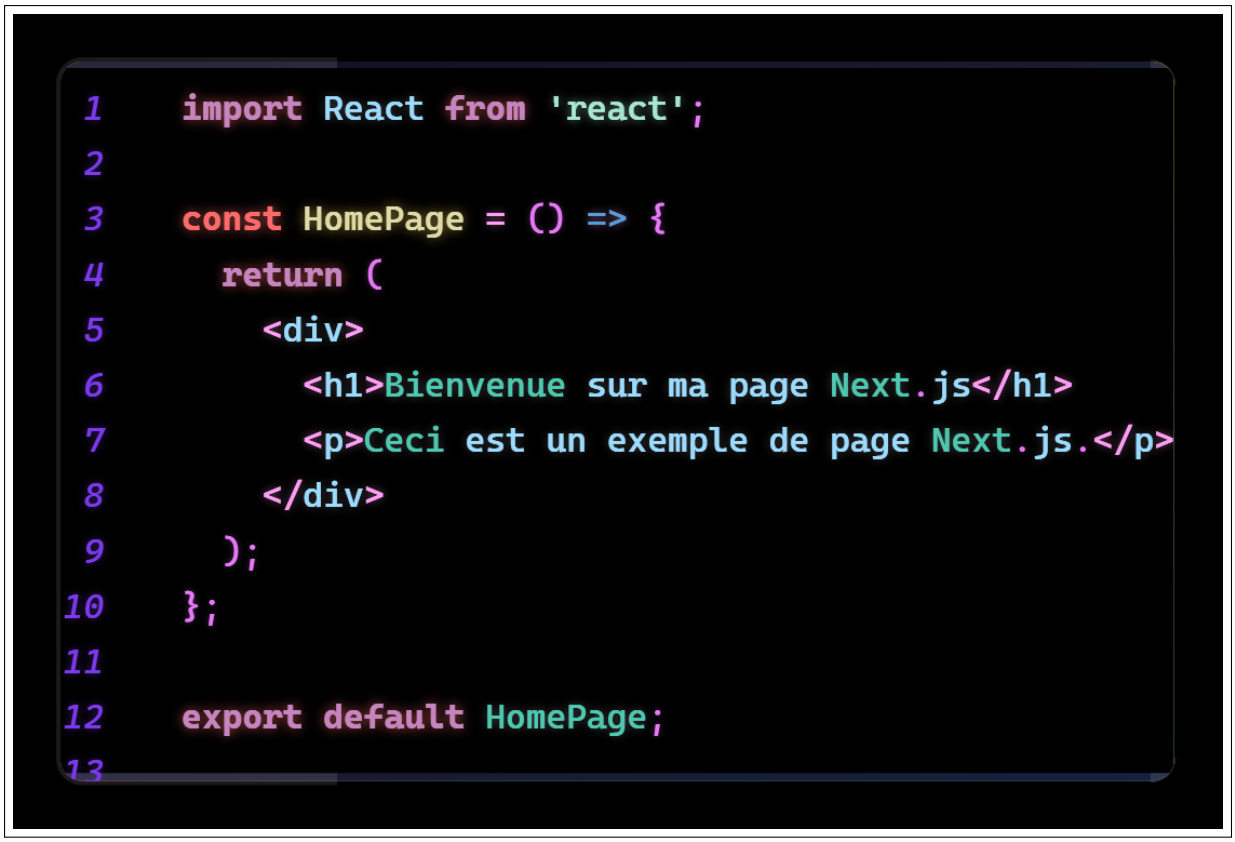
I C'est quoi Next.js ?

1 Définition et intérêt

Next.js est un framework React qui permet de construire des applications web modernes avec une expérience utilisateur améliorée. Il offre des fonctionnalités telles que le rendu côté serveur (SSR), la génération de sites statiques (SSG), et l'hydratation, ce qui permet de créer des applications web rapides et optimisées pour le référencement (SEO).

1.1 Exemple d'utilisation de Next.js

Voici un exemple simple d'une page Next.js :



```jsx
import React from 'react';

const HomePage = () => {
  return (
    <div>
      <h1>Bienvenue sur ma page Next.js</h1>
      <p>Ceci est un exemple de page Next.js.</p>
    </div>
  );
};

export default HomePage;
```

FIGURE 1.1 – Exemple d'utilisation de Next.js

2 SSR, SSG et ISR

2.1 SSR (Server-Side Rendering)

Le rendu côté serveur permet de générer le contenu de la page sur le serveur à chaque requête. Cela améliore les performances et le SEO.

2.2 SSG (Static Site Generation)

La génération de sites statiques permet de pré-rendre les pages au moment de la construction. Cela est idéal pour les pages qui ne changent pas fréquemment.

2.3 ISR (Incremental Static Regeneration)

L'ISR permet de mettre à jour les pages statiques après la construction, sans avoir à reconstruire l'ensemble du site.

3 API REST

Next.js permet également de créer des API REST directement dans votre application. Vous pouvez créer des endpoints API dans le dossier pages/api.

3.1 Exemple d'API REST avec Next.js

Voici un exemple simple d'un endpoint API :

FIGURE 1.2 – Exemple d'API REST avec Next.js

II Technologies qui vont être expliquées dans ce cours

1 Frameworks principaux

1.1 React.js

React.js est une bibliothèque JavaScript pour construire des interfaces utilisateur. Elle permet de créer des composants réutilisables et de gérer l'état de l'application de manière efficace.

1.2 Next.js

Next.js est un framework basé sur React qui offre des fonctionnalités avancées pour le rendu côté serveur, la génération de sites statiques, et bien plus encore.

1.3 Express.js

Express.js est un framework minimaliste pour Node.js qui permet de créer des applications web et des API rapidement et facilement.

1.4 Electron.js

Electron.js est un framework pour créer des applications de bureau multiplateformes avec JavaScript, HTML, et CSS.

2 Styling et Cascading

2.1 TailwindCSS

TailwindCSS est un framework CSS utilitaire qui permet de styliser rapidement et efficacement vos applications web.

2.2 GSAP

GSAP (GreenSock Animation Platform) est une bibliothèque JavaScript pour créer des animations performantes et fluides.

2.3 Three.js

Three.js est une bibliothèque JavaScript qui permet de créer des graphiques 3D dans le navigateur.

3 Authentification et sessions

3.1 Bcrypt

Bcrypt est une bibliothèque pour hacher les mots de passe de manière sécurisée.

3.2 JsonWebToken

JsonWebToken (JWT) est un standard pour créer des jetons d'authentification qui peuvent être vérifiés et approuvés de manière sécurisée.

3.3 Google-Auth-Library

Google-Auth-Library est une bibliothèque pour authentifier les utilisateurs avec Google.

3.4 Re-Captcha

Re-Captcha est un service de Google pour protéger vos sites web contre les spams et les abus.

3.5 Cookie.js

Cookie.js est une bibliothèque pour gérer les cookies dans le navigateur.

4 Génération de documents

4.1 Puppeteer

Puppeteer est une bibliothèque Node.js pour contrôler un navigateur headless Chrome ou Chromium. Elle est souvent utilisée pour générer des PDF ou des captures d'écran.

4.2 Excel.js

Excel.js est une bibliothèque pour lire, écrire et manipuler des fichiers Excel en JavaScript.

5 Communication en temps réel

5.1 Socket.IO

Socket.IO est une bibliothèque pour la communication en temps réel entre le client et le serveur. Elle permet de créer des applications web interactives et dynamiques.

5.2 WebRTC

WebRTC (Web Real-Time Communication) est une technologie qui permet la communication en temps réel entre les navigateurs, y compris la voix, la vidéo et le partage de données.

6 SMTP

6.1 Nodemailer

Nodemailer est une bibliothèque pour envoyer des emails en Node.js. Elle prend en charge plusieurs services de messagerie et est facile à utiliser.

6.2 SendGrid / Brevo

SendGrid et Brevo sont des services de messagerie transactionnelle qui permettent d'envoyer des emails à grande échelle de manière fiable et sécurisée.

III Préparation de l'environnement et installations

1 Installation de Node.js

Pour commencer à travailler avec Next.js, vous devez d'abord installer Node.js. Vous pouvez télécharger et installer Node.js depuis le site officiel Node.js.

1.1 Vérification de l'installation

Après avoir installé Node.js, vous pouvez vérifier l'installation en ouvrant un terminal et en exécutant les commandes suivantes :

```
node -v
```

Ces commandes devraient afficher les versions de Node.js et npm installées sur votre système.

2 Création du premier projet Next.js

Pour créer votre premier projet Next.js, vous pouvez utiliser l'outil de ligne de commande **create-next-app**, qui vous permet de configurer rapidement un nouveau projet. Voici les étapes à suivre :

1. **Ouvrir un terminal** : Vous pouvez utiliser le terminal de votre système d'exploitation ou un terminal intégré dans votre éditeur de code.
2. **Exécuter la commande de création** : Utilisez la commande suivante pour créer un nouveau projet Next.js :

   ```
   npx create-next-app@latest mon-premier-projet
   ```

   Remplacez **mon-premier-projet** par le nom que vous souhaitez donner à votre projet.

3. **Suivre les instructions** : L'outil vous posera quelques questions pour configurer votre projet. Vous pouvez accepter les options par défaut en appuyant sur Entrée.
4. **Accéder au répertoire du projet** : Une fois le projet créé, accédez au répertoire du projet avec la commande :

```
cd mon-premier-projet
```

5. **Démarrer le serveur de développement** : Pour démarrer le serveur de développement et voir votre application en action, utilisez la commande :

```
npm run dev
```

6. **Ouvrir l'application dans le navigateur** : Par défaut, votre application sera accessible à l'adresse **http ://localhost :3000**. Ouvrez cette URL dans votre navigateur pour voir votre nouveau projet Next.js.

Félicitations ! Vous avez maintenant créé et démarré votre premier projet Next.js. Vous pouvez commencer à explorer les fichiers et à modifier le code pour apprendre comment fonctionne Next.js.

3 Bases de données

3.1 MySQL - XAMPP

XAMPP est un environnement de développement local qui inclut MySQL, Apache, PHP et Perl. Vous pouvez télécharger XAMPP depuis le site officiel Apache Friends.

3.2 MongoDB - MongoDB Compass

MongoDB Compass est une interface graphique pour MongoDB. Vous pouvez télécharger MongoDB Compass depuis le site officiel MongoDB Compass.

3.3 MongoDB - MongoDB Atlas

MongoDB Atlas est un service de base de données cloud pour MongoDB. Vous pouvez créer un compte et configurer une base de données sur MongoDB Atlas.

Chapitre 2 : Guide en React.js pour passer à Next.js

Chapitre 2

Guide en React.js pour passer à Next.js

I Rappels en JavaScript

1 Destructeurs, Mapping, Filtrage, et Réducteurs des tableaux

Les destructeurs permettent de décomposer des structures de données comme les tableaux ou les objets en variables distinctes.

Exemple de destructuration :

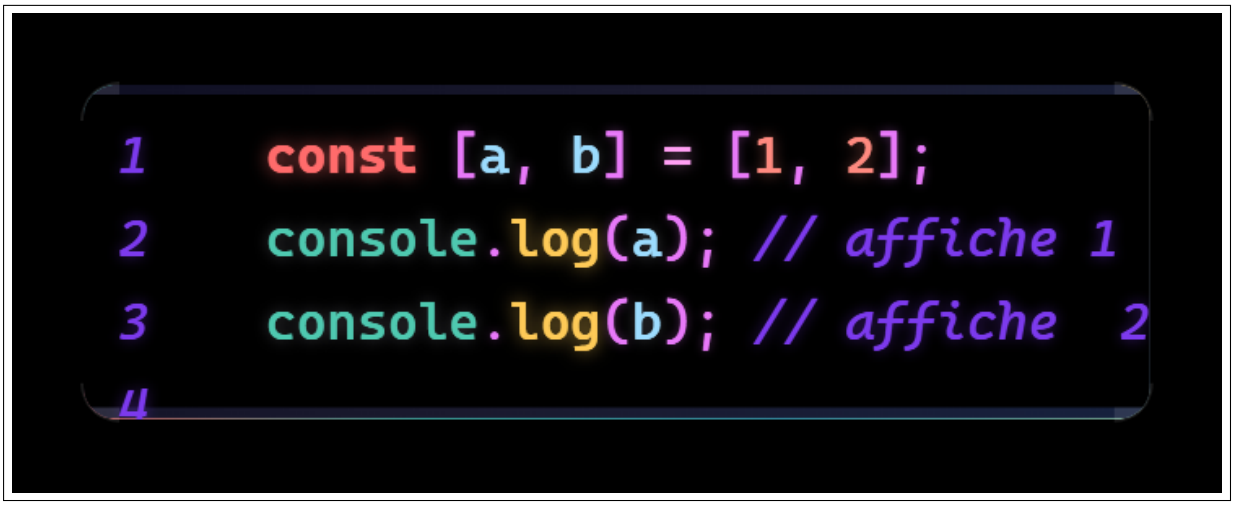


FIGURE 2.1 – Exemple de destructuration

Le mapping, le filtrage et la réduction sont des méthodes courantes pour manipuler les tableaux.

Exemple de mapping :

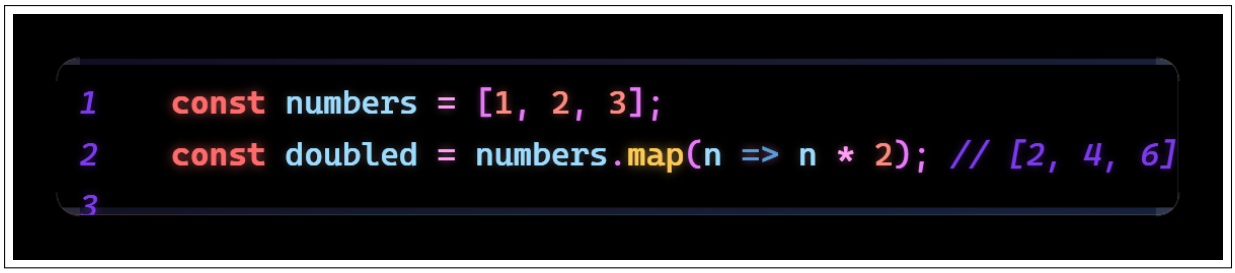


FIGURE 2.2 – Exemple de mapping

2 Promesses et fonctions asynchrones

Les promesses sont utilisées pour gérer les opérations asynchrones en JavaScript. Une promesse représente une valeur qui peut être disponible maintenant, ou à l'avenir, ou jamais.

Exemple de promesse :

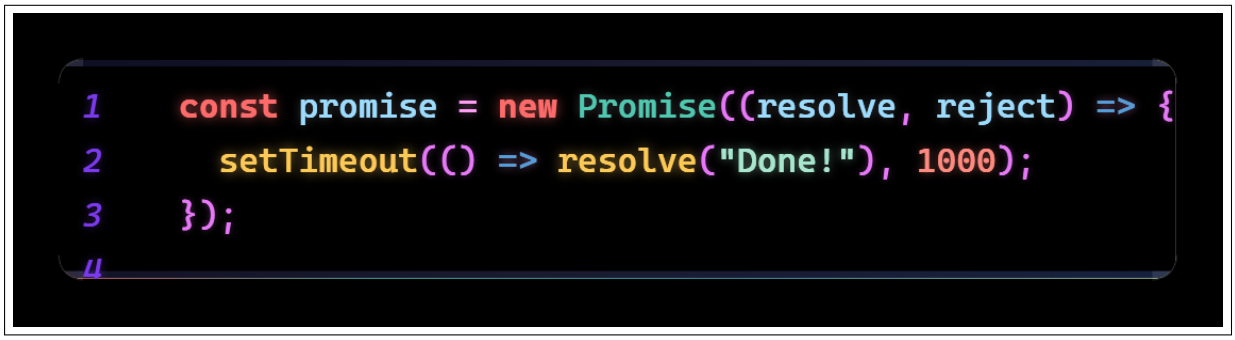


FIGURE 2.3 – Exemple de promesse

Les fonctions asynchrones permettent d'écrire du code asynchrone qui ressemble à du code synchrone.

Exemple de fonction asynchrone :

FIGURE 2.4 – Exemple de fonction asynchrone

3 Modification d'un tableau d'objets avec des fonctions

Pour modifier un tableau d'objets, on peut utiliser des fonctions comme map pour appliquer des transformations.

Exemple de modification d'un tableau d'objets :

```
const users = [{ name: 'Alice', age: 25 }, { name: 'Bob', age: 30 }];
const updatedUsers = users.map(user => ({ ...user, age: user.age + 1 }));

```

FIGURE 2.5 – Exemple de modification d'un tableau d'objets

4 Copier un texte dans le clipboard

Pour copier du texte dans le presse-papiers, on peut utiliser l'API Clipboard.
Exemple de copie dans le presse-papiers :

```
function copyToClipboard(text) {
  navigator.clipboard.writeText(text).then(() => {
    console.log('Text copied to clipboard');
  });
}

```

FIGURE 2.6 – Exemple de copie dans le presse-papiers

5 Détection de la voix et la transformer en texte

La détection de la voix et sa transformation en texte peuvent être réalisées avec l'API Web Speech.
Exemple de détection de la voix :

```
const recognition = new webkitSpeechRecognition();
recognition.onresult = function(event) {
  const speechToText = event.results[0][0].transcript;
  console.log(speechToText);
};
recognition.start();

```

FIGURE 2.7 – Exemple de détection de la voix

II La notion du JSX

1 Introduction au JSX

Le JSX est une extension de syntaxe pour JavaScript, il ressemble beaucoup au HTML, mais il est transformé en appels de fonctions JavaScript par des outils comme Babel. Cela permet aux développeurs de créer des interfaces utilisateur de manière déclarative.

Exemple de JSX :

```
1    const element = <h1>Bonjour, monde!</h1>;
2
```

FIGURE 2.8 – Exemple de JSX

2 Différences entre JSX et HTML

Bien que le JSX ressemble à HTML, il existe quelques différences clés :

2.1 Attributs

En JSX, certains attributs HTML sont renommés pour suivre la convention de nommage de JavaScript. Par exemple, l'attribut **class** devient **className**, et **for** devient **htmlFor**, car**class** et **for** sont déjà des noms réservés dans code JavaScript.

Exemples d'attributs JSX :

```
1    <label htmlFor="name">Nom :</label>
2    <input type="text" id="name" className="input-field" />
3
```

FIGURE 2.9 – Exemples d'attributs JSX

2.2 Événements

En JSX, les gestionnaires d'événements sont nommés en utilisant le camelCase et sont souvent passés comme des fonctions JavaScript. Par exemple, **onclick** devient **onClick**.

Exemple d'événements :

FIGURE 2.10 – Exemple d'événements

Remarque : Il faut faire attention aux fonctions passées aux événements :

- **onClick={ma_fonction} :** pour appeler une fonction déjà définie quand l'événement est declanché.
- **onClick={() => alert("Salut")} :** pour définir et appeler une fonction quand l'événement est declanché.
- **onClick={alert("Salut")} :** Fausse utilisation ! Comme ça la fonction s'appelle toujours, on aura une alerte avec "Salut" même si on n'a pas cliqué sur le bouton.

2.3 Expressions JavaScript

Le JSX permet d'intégrer des expressions JavaScript directement dans le code en les entourant d'accolades **{ }**.

Exemple d'expressions JavaScript :

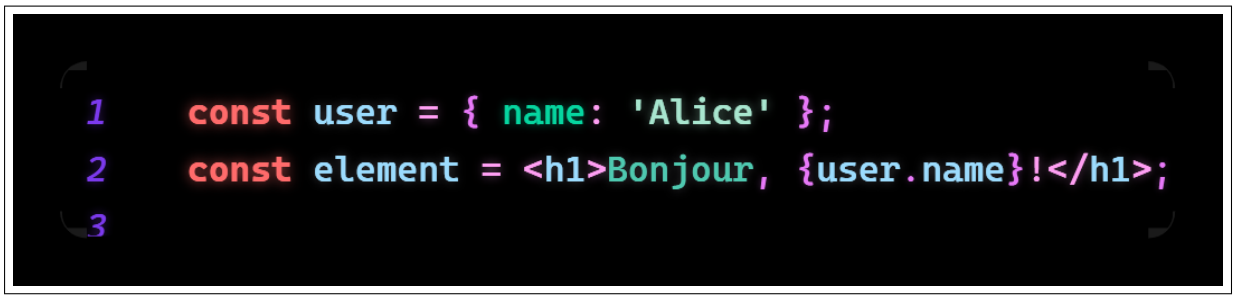


FIGURE 2.11 – Exemple d'expressions JavaScript

3 Routes des pages

Next.js utilise un système de routage basé sur les fichiers, ce qui signifie que les routes sont déterminées par la structure des fichiers dans le dossier `pages`.

3.1 Structure de base

Chaque fichier JavaScript dans le dossier **pages** devient automatiquement une route accessible. Par exemple, un fichier nommé **About.js** dans le dossier **pages** sera accessible via l'URL **/About**.

Exemple de structure de fichiers :

```
pages/
  _app.jsx       $\rightarrow$ Route: non accessible, contient des configurations, etc
  index.jsx      $\rightarrow$ Route: /
```

```
About.jsx       $\rightarrow$ Route: /about
Contact.jsx     $\rightarrow$ Route: /contact
```

_app.jsx , Ce fichier contient :

```jsx
export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />;
}
```

FIGURE 2.12 – Fichier principal

III Le rendring en React

1 Structure d'une page

Une page React est un fichier composé de 4 parties :

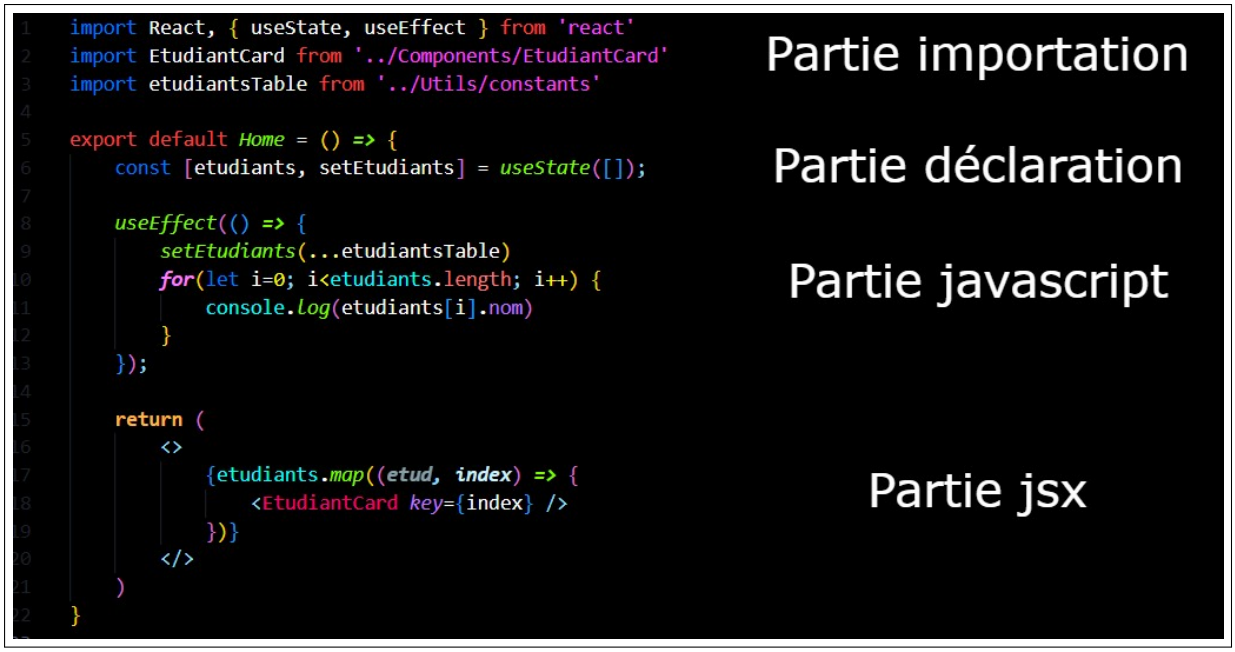


FIGURE 2.13 – Structure d'un fichier

2 Components et Props

Une page en React est divisée en plusieurs components dépendants ou indépendants, ils sont importés dans un même fichier pour construire notre page tout en gardant le code bien structuré, le header séparé, le footer séparé, le sidebar aussi, etc... Cela facilite la modification et le débugging.

Exemple de structure d'une page avec les components :

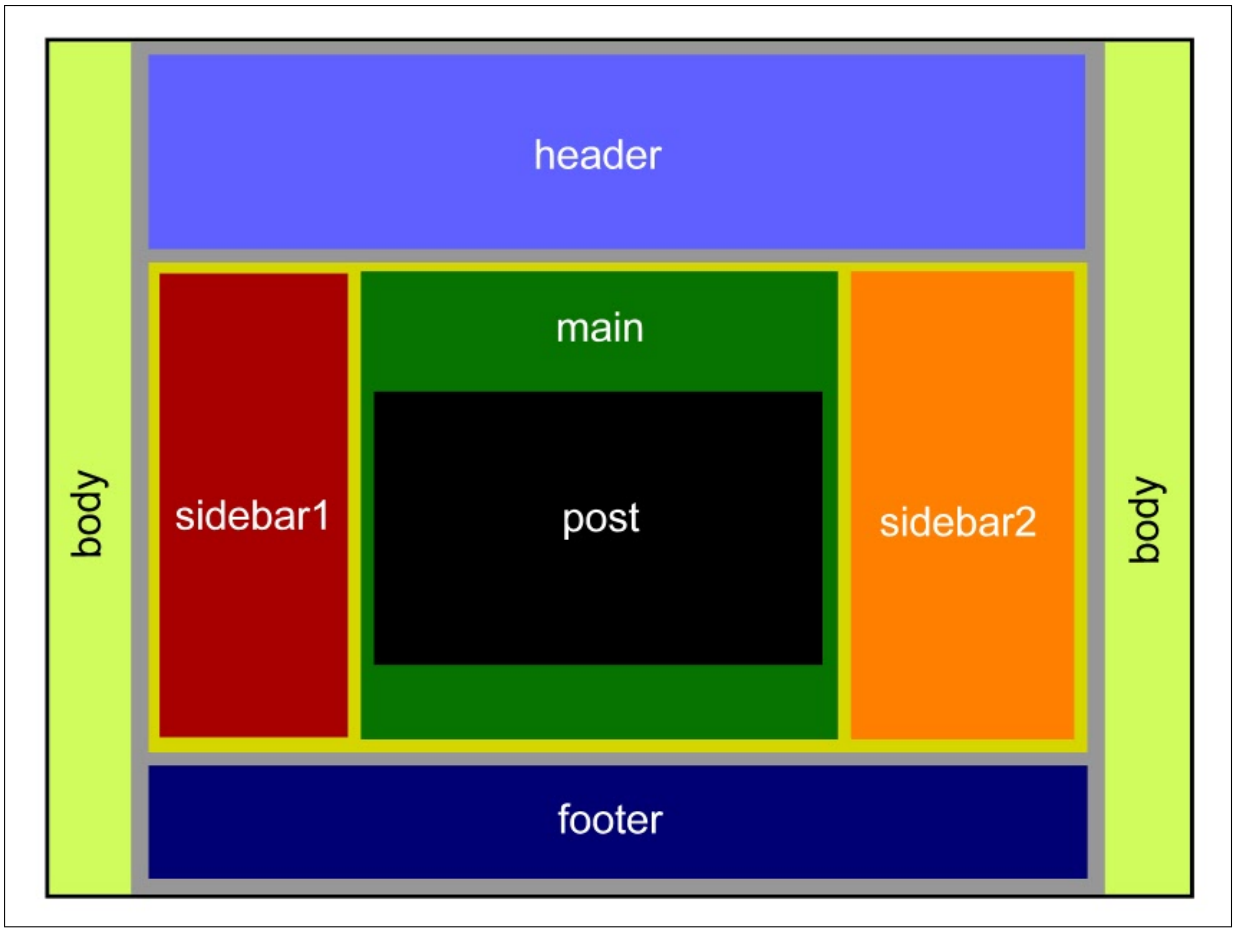


FIGURE 2.14 – Components

Les props sont des **"Paramètres"** passés à un component qui contient des informations, par exemple si on veut un header affichant **"Salut Monsieur <nom>"** avec le nom est passé dynamiquement au header, on fait :

Dans le dossier des pages :

```
1   import Header from '..Components/Header"
2
3   function Main() {
4     const person_name = "Pahae";
5
6     return (
7       <div>
8         <Header name={person_name} />
9         <p>Voici le corps de notre siteweb</p>
10      </div>
11    );
12  }
13
```

FIGURE 2.15 – Page principale à importer le component

Dans le dossier des components :

```
export default function Header({ props }) {
  return (
    <div>
      <p>Salut Monsieur {props.name}</p>
    </div>
  );
}

```

FIGURE 2.16 – Exemple de component

Importation / Exportation : Next.js permet d'importer et d'exporter des composants et des modules pour une meilleure organisation du code.

- Importation par défaut :

```
import n_importe_quoi from '../components/MyComponent'; // Si le MyComponent contient un "export default"

```

FIGURE 2.17 – Importation par défaut

- Importation nommée :

```
import MyComponent from '../components/MyComponent'; // Si le MyComponent contient un "export"

```

FIGURE 2.18 – Importation nommée

3 Fragments

Ce code là génère une grande erreur :

FIGURE 2.19 – Erreur fragments

Car React ne permet pas de faire retourner plus qu'un seul élément par une fonction composante, pour cela on utilise les **fragments** pour regrouper les éléments, voic un exemple :

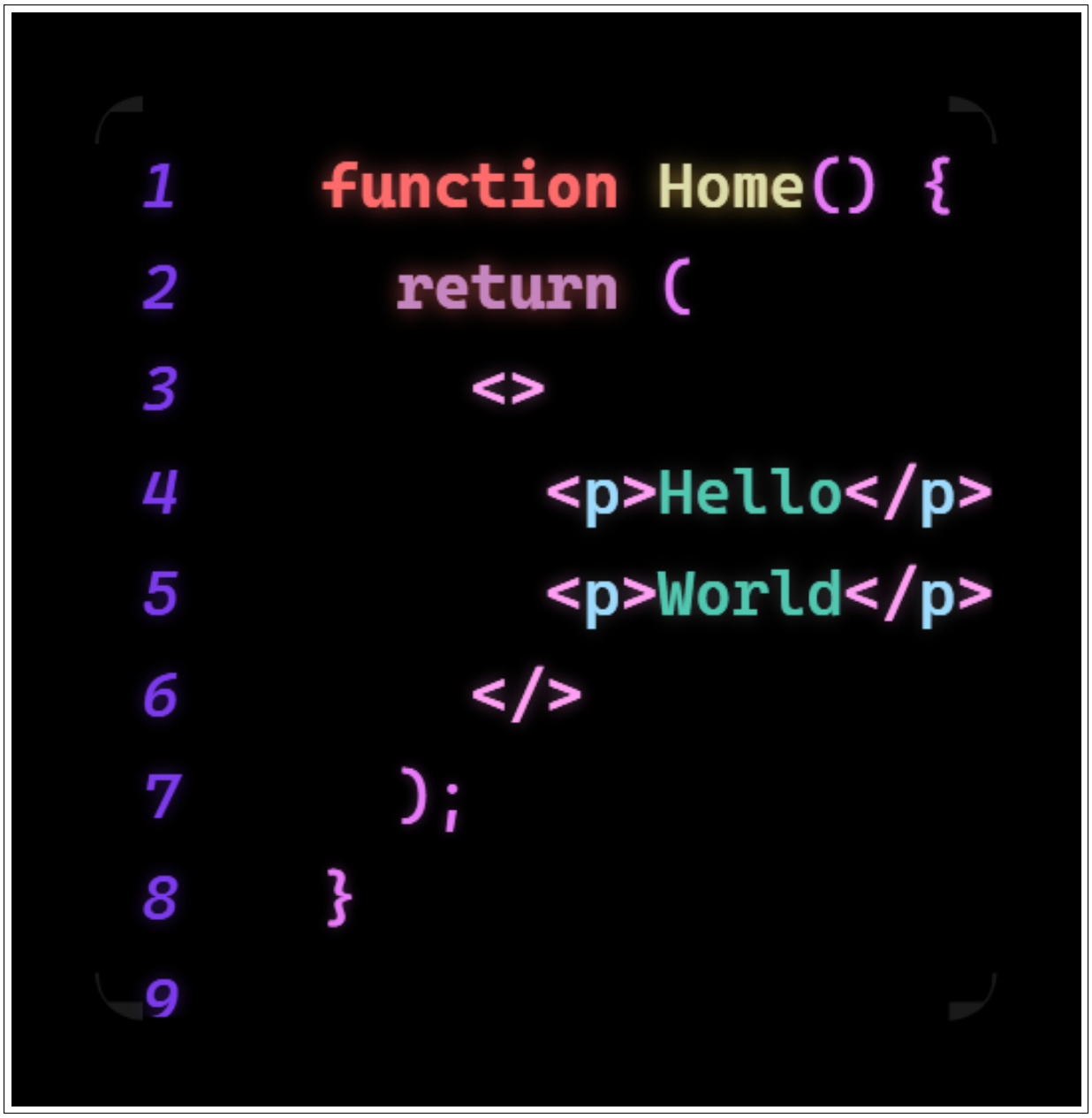


FIGURE 2.20 – Exemple fragments

4 Conditional rendring

Le rendu conditionnel en React fonctionne de la même manière que les conditions en JavaScript. On utilise des instructions **if** ou des opérateurs conditionnels pour créer des éléments représentant l'état actuel et laisser React mettre à jour l'interface utilisateur en conséquence.

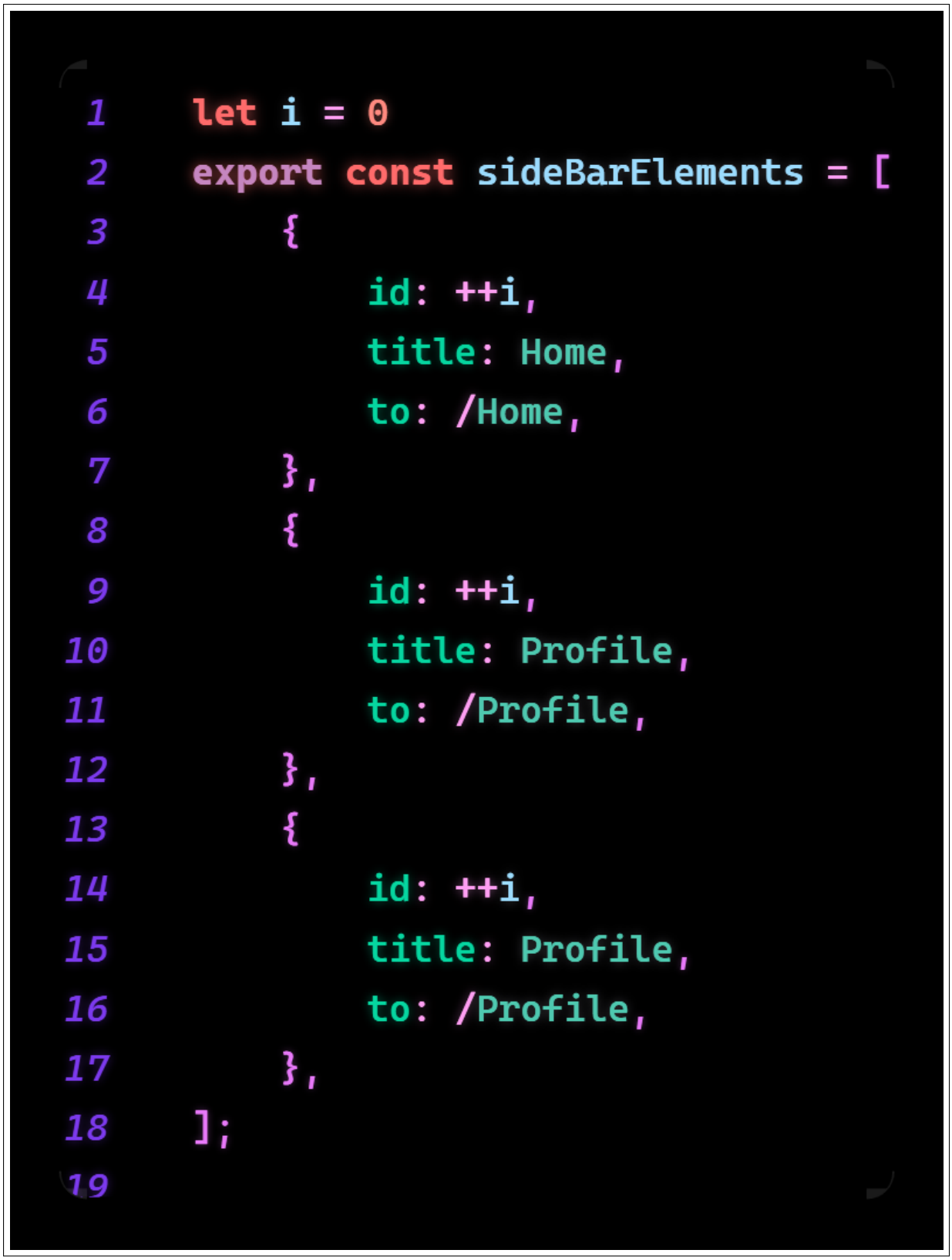
Exemple de rendu conditionnel :

```
function Profile(props) {
  const isLoggedIn = props.isLoggedIn;
  if (isLoggedIn) {
    return <Home />;
  }
  else {
    return <Login />;
  }
}
```

FIGURE 2.21 – Exemple de rendu conditionnel

5 Fichier des constantes

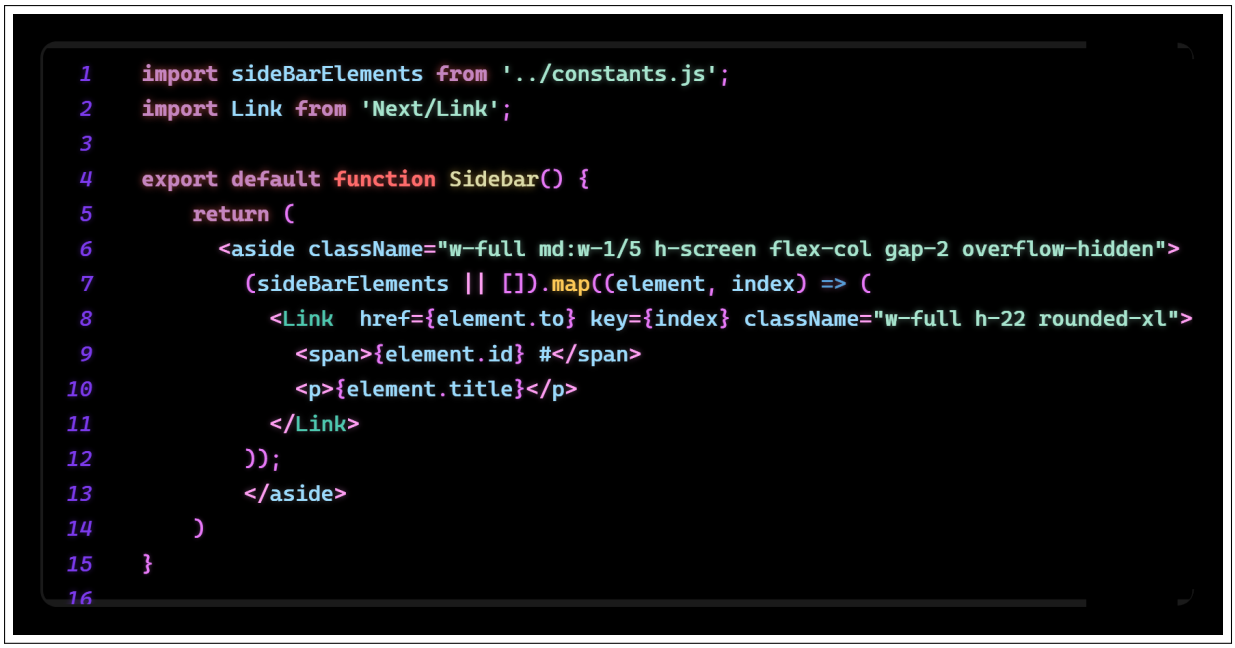
Un fichier de constantes permet de stocker des valeurs constantes utilisées dans toute l'application, comme les URLs des API ou les configurations.



```
let i = 0
export const sideBarElements = [
    {
        id: ++i,
        title: Home,
        to: /Home,
    },
    {
        id: ++i,
        title: Profile,
        to: /Profile,
    },
    {
        id: ++i,
        title: Profile,
        to: /Profile,
    },
];

```

FIGURE 2.22 – Fichier des constantes

Dans constants.js :



```jsx
import sideBarElements from '../constants.js';
import Link from 'Next/Link';

export default function Sidebar() {
    return (
      <aside className="w-full md:w-1/5 h-screen flex-col gap-2 overflow-hidden">
        (sideBarElements || []).map((element, index) => (
          <Link  href={element.to} key={index} className="w-full h-22 rounded-xl">
            <span>{element.id} #</span>
            <p>{element.title}</p>
          </Link>
        ));
        </aside>
    )
}
```

FIGURE 2.23 – Importation du fichier des constantes

Dans Components/Sidebar.jsx :

IV La notion des Hooks

1 Variables d'état : useState

Le Hook **useState** permet d'ajouter un état local à des composants fonctionnels. Il retourne une paire : la valeur actuelle de l'état et une fonction pour la mettre à jour.

Exemple de `useState` :

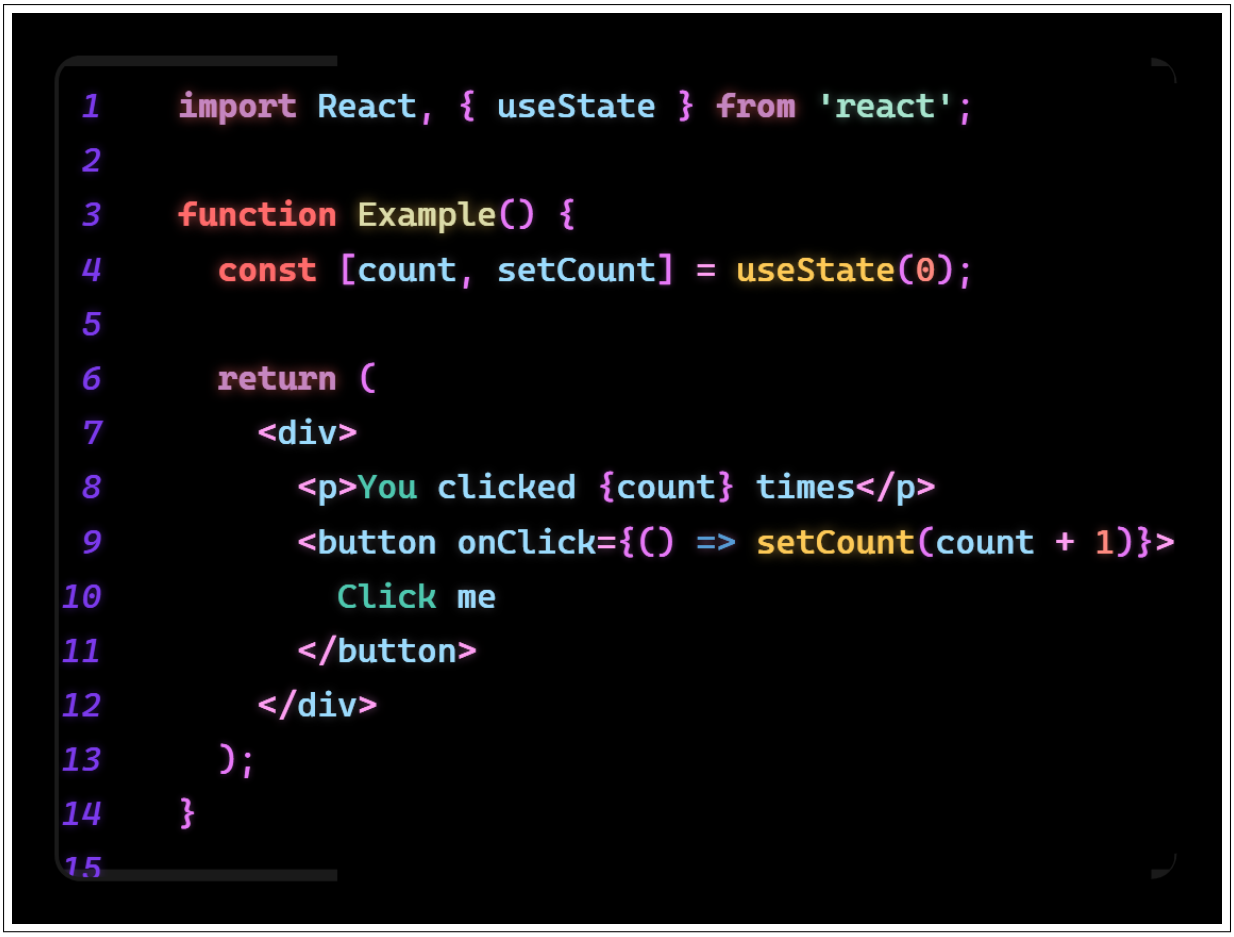


FIGURE 2.24 – useState

2 Effets secondaires : useEffect / useLayoutEffect

2.1 useEffect

Le Hook `useEffect` permet de réaliser des effets secondaires dans des composants fonctionnels, comme la récupération de données, la mise à jour du DOM, et l'abonnement à des événements.

Exemple de `useEffect` :

```jsx
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

```

FIGURE 2.25 – useEffect

2.2 Différence entre useEffect et useLayoutEffect

useEffect

- **Quand ?** Après que tout est affiché à l'écran (le rendu est terminé).
- **À quoi ça sert ?** Pour faire des actions après l'affichage, comme : récupérer des données (API), écouter un événement...
- **Avantage ?** Il ne bloque pas le rendu de la page, donc elle s'affiche plus vite.

useLayoutEffect

- **Quand ?** Juste avant que la page soit affichée à l'écran.
- **À quoi ça sert ?** Pour faire des actions qui modifient le layout (la mise en page) avant que l'utilisateur voie quoi que ce soit. Par exemple : mesurer la taille d'un élément...
- **Avantage ?** Le changement est invisible pour l'utilisateur.

Résumé

- useEffect = après affichage → l'utilisateur voit la page puis l'effet s'exécute.
- useLayoutEffect = avant affichage → l'effet s'exécute avant que quoi que ce soit ne soit visible.

2.3 Tableau des dépendances

Les Hooks **useEffect** et **useLayoutEffect** sont utilisés pour gérer les effets secondaires dans les composants fonctionnels de React. Ils prennent deux paramètres principaux : une fonction de rappel (callback) et un tableau de dépendances.

Le tableau des dépendances est un tableau qui contient les variables que la fonction de rappel utilise. React utilise ce tableau pour déterminer quand ré-exécuter l'effet. Si l'une des dépendances change entre les rendus, la fonction de rappel est ré-exécutée.

Comportements selon le tableau des dépendances

— **Tableau vide** : Si vous passez un tableau vide ([]), l'effet ne s'exécutera qu'une seule fois, après le premier rendu. Cela équivaut à **componentDidMount** dans les composants de classe.

```
useEffect(() => {
  console.log('Exécuté une seule fois après le premier rendu.');
}, []);
```

— **Tableau avec dépendances** : Si vous incluez des variables dans le tableau, l'effet s'exécutera à chaque fois que l'une de ces variables changera. Cela permet de réagir aux mises à jour des props ou de l'état.

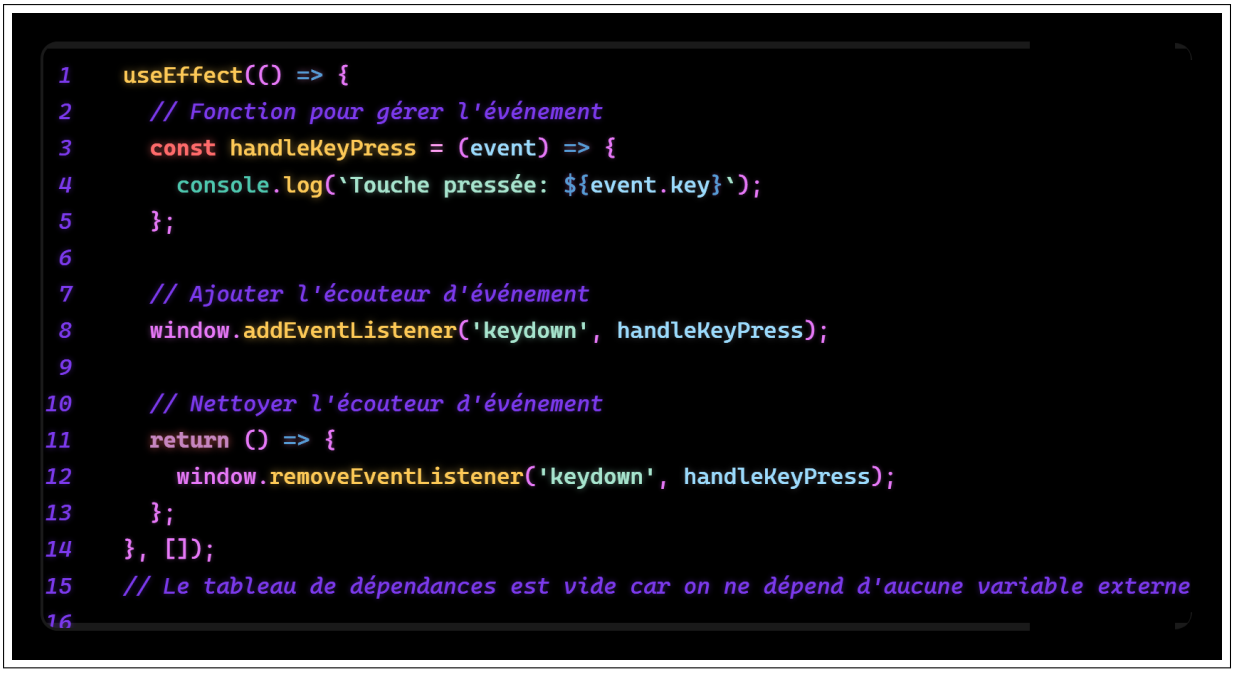
```
useEffect(() => {
  console.log('Exécuté à chaque fois que `x` ou `y` change.');
}, [x, y]);
```

— **Aucun tableau** : Si vous n'incluez pas de tableau de dépendances, l'effet s'exécutera à chaque rendu du composant. Cela peut entraîner des performances médiocres et n'est généralement pas recommandé.

```
useEffect(() => {
  console.log('Exécuté à chaque rendu du composant.');
});
```

Nettoyage des effets Si votre effet nécessite un nettoyage (comme annuler un abonnement ou supprimer un écouteur d'événement), vous pouvez retourner une fonction de nettoyage depuis la fonction de rappel. Cette fonction sera exécutée avant que le composant ne soit démonté ou avant que l'effet ne soit ré-exécuté.

Exemple de nettoyage d'effet :



```
useEffect(() => {
  // Fonction pour gérer l'événement
  const handleKeyPress = (event) => {
    console.log(`Touche pressée: ${event.key}`);
  };

  // Ajouter l'écouteur d'événement
  window.addEventListener('keydown', handleKeyPress);

  // Nettoyer l'écouteur d'événement
  return () => {
    window.removeEventListener('keydown', handleKeyPress);
  };
}, []);
// Le tableau de dépendances est vide car on ne dépend d'aucune variable externe
```

FIGURE 2.26 – Exemple de nettoyage d'effet

En utilisant correctement le tableau des dépendances, vous pouvez optimiser les performances de vos composants React et éviter des effets secondaires inutiles. Cela permet de s'assurer que vos effets ne s'exécutent que lorsque cela est nécessaire, en fonction des changements des dépendances spécifiées.

3 Références : useRef

Le Hook **useRef** est utilisé pour créer une référence à un élément du DOM ou pour stocker une valeur mutable qui persiste pendant toute la durée de vie d'un composant. Imaginez-le comme une boîte où vous pouvez stocker quelque chose et y accéder plus tard sans déclencher de re-rendu du composant.

3.1 Accéder à un élément du DOM

L'une des utilisations les plus courantes de **useRef** est de se concentrer sur un champ de saisie. Par exemple, vous pouvez avoir un bouton qui, lorsqu'il est cliqué, met le focus sur un champ de texte.

Voici un exemple simple :

```jsx
import React, { useRef } from 'react';

// Composant avec un champ de texte et un bouton
export default function Home() {
  // Créer une référence pour le champ de texte
  const inputRef = useRef(null);

  // Fonction pour mettre le focus sur le champ de texte
  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <>
      {/* Champ de texte avec la référence */}
      <input ref={inputRef} type="text" />

      {/* Bouton pour mettre le focus sur le champ de texte */}
      <button onClick={focusInput}>Focus sur le champ de texte</button>
    </>
  );
}
```

FIGURE 2.27 – useRef

4 Valeurs mémorisées : useMemo / useCallback

4.1 useMemo

Le Hook **useMemo** permet de mémoriser des valeurs calculées pour éviter des calculs coûteux à chaque rendu.

Soit la fonction **calculerBudget** qui effectue un calcul très complexe et retourne un résultat :

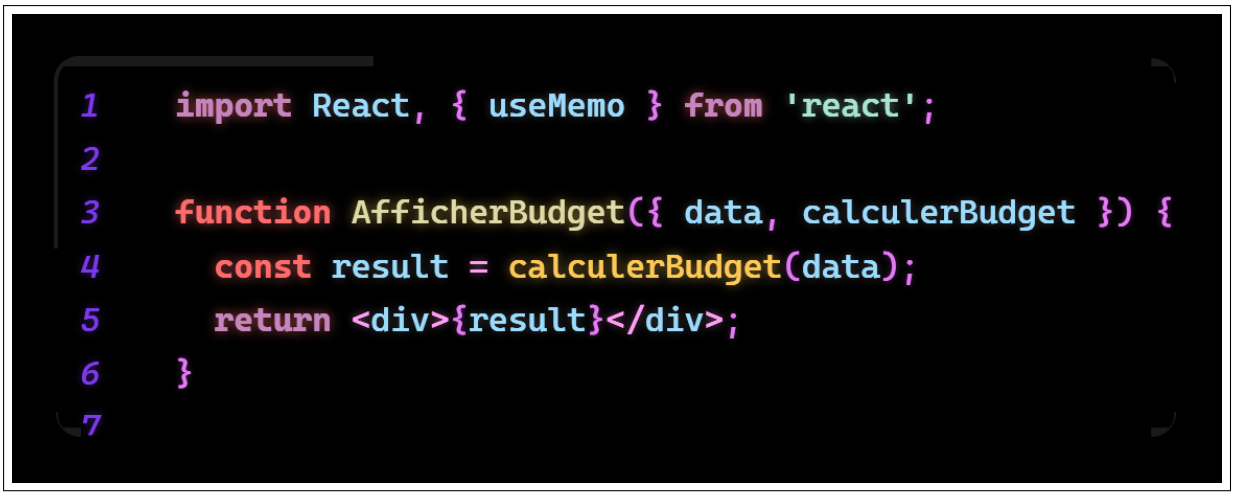


FIGURE 2.28 – calculerBudget

Si le component **AfficherBudget.jsx** se re-render souvent, ce calcul sera refait à chaque fois, même si data n'a pas changé. Mais avec useMemo, le calcul ne sera refait que si data change. Sinon, React réutilise le résultat précédent. Voici un exemple optimier comment avec **useMemo** :

```
import React, { useMemo } from 'react';

function AfficherBudget({ data, calculerBudget }) {
  const result = useMemo(() => calculerBudget(data), [data]);
  return <div>{result}</div>;
}
```

4.2 useCallback

Le Hook **useCallback** retourne une fonction mémorisée. Il est utile lorsque vous avez besoin de passer une fonction comme prop à des composants enfants optimisés qui dépendent de l'égalité des références pour éviter des rendus inutiles.

Exemple de **useCallback** :

```
import React, { useCallback } from 'react';

function ParentComponent() {
  const [count, setCount] = React.useState(0);

  const increment = useCallback(() => {
    setCount(c => c + 1);
  }, [setCount]);

  return <ChildComponent onIncrement={increment} />;
}
```

5 Identifiant unique : useId

Le Hook **useId** est utilisé pour générer des identifiants uniques qui peuvent être passés aux attributs **id** des éléments et aux attributs **aria-***.

Exemple de **useId** :

```
import React, { useId } from 'react';

function Checkbox() {
  const id = useId();
  return (
    <div>
      <label htmlFor={id}>Do you like React?</label>
      <input id={id} type="checkbox" name="react"/>
    </div>
  );
}
```

Remarque : l'ID va reste stable pendant toute la durée de vie du composant côté client (tant que la page n'est pas rechargée).

V Styling en React

1 Global styles

Les styles globaux sont appliqués à toute l'application. Ils sont généralement définis dans un fichier CSS global qui est importé dans l'application.

Exemple de styles globaux :

Dans styles/globals.css :

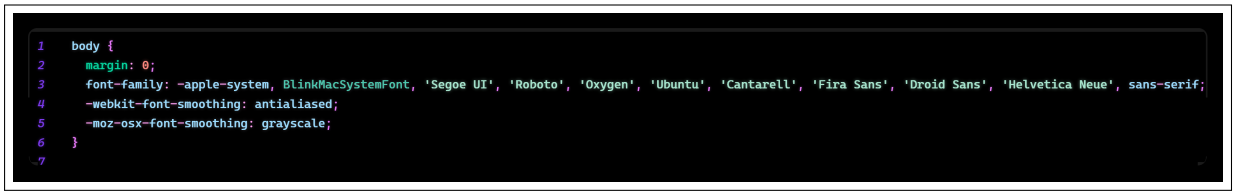


FIGURE 2.29 – globalscss

Dans pages/_app.jsx :

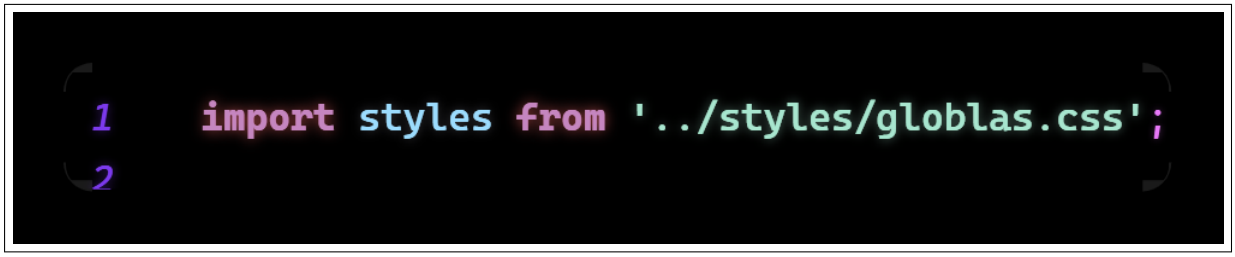


FIGURE 2.30 – Importation de globalscss

2 Modular styles

Les styles modulaires permettent de définir des styles qui ne s'appliquent qu'à un composant spécifique. Cela peut être réalisé en utilisant des modules CSS.

Exemple de styles modulaires :

```
1  import styles from './Button.module.css';
2
3  function Button() {
4    return <button className={styles.button}>Click me</button>;
5  }
6
```

FIGURE 2.31 – Exemple de styles modulaires

3 Internal styles

Les styles internes sont définis directement dans les fichiers JavaScript en utilisant des objets JavaScript.

Exemple de styles internes :

```
1  const styles = {
2    button: {
3      backgroundColor: 'blue',
4      color: 'white',
5      padding: '10px 20px',
6      border: 'none',
7      borderRadius: '5px',
8    }
9  };
10
11 function Button() {
12   return <button style={styles.button}>Click me</button>;
13 }
14
```

FIGURE 2.32 – Exemple de styles internes

4 Inline styles

Les inline styles permettent de définir des styles directement dans le JSX en utilisant l'attribut **style**.

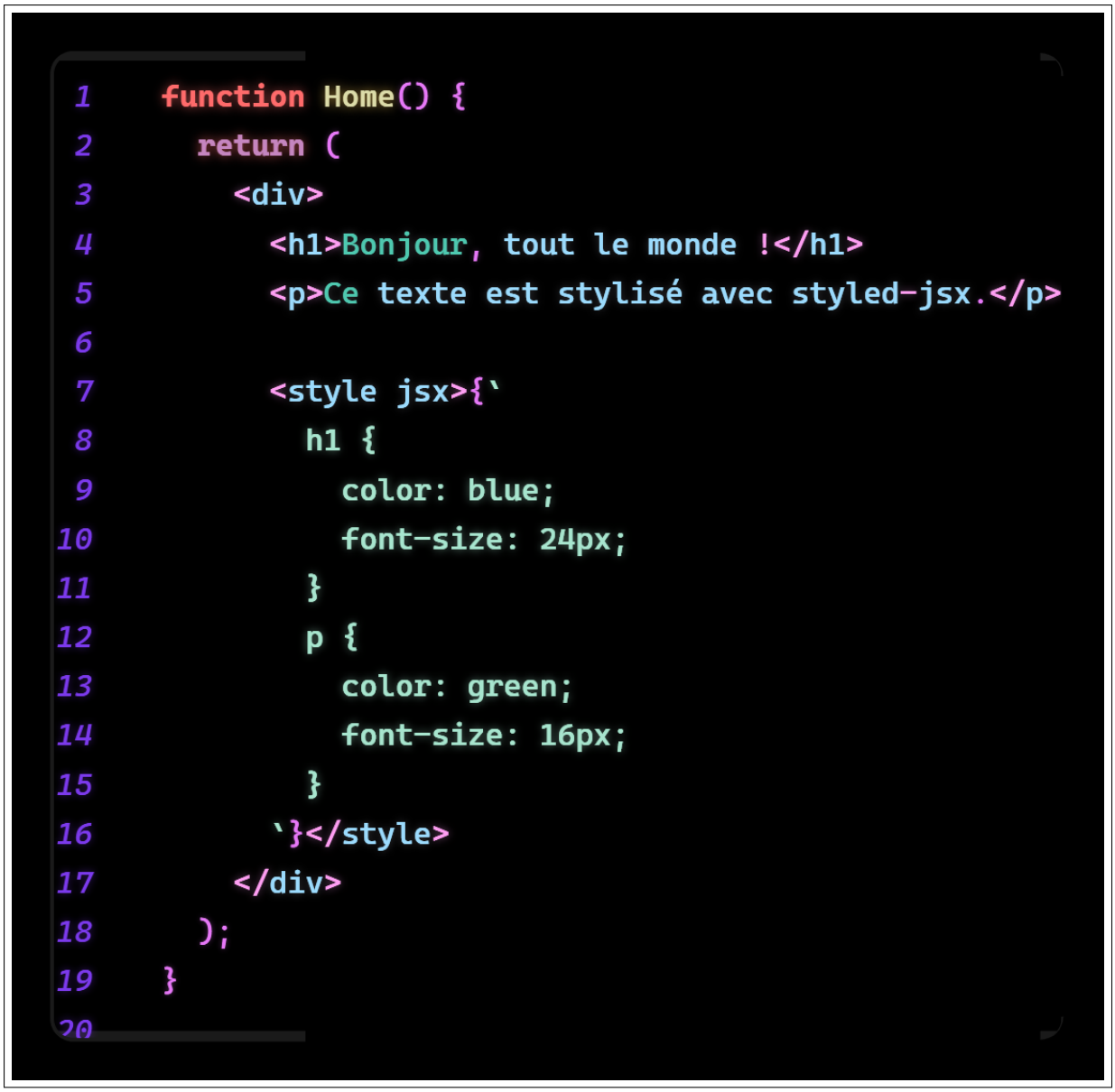
Exemple de inline styles :

FIGURE 2.33 – Exemple de inline styles

5 JSX styles

Dans Next.js, vous pouvez utiliser la balise **<style jsx>** pour définir des styles CSS directement dans vos composants. Cela permet d'encapsuler les styles afin qu'ils ne s'appliquent qu'au composant actuel, évitant ainsi les conflits de styles avec d'autres composants.

Exemple de styles :



```
function Home() {
  return (
    <div>
      <h1>Bonjour, tout le monde !</h1>
      <p>Ce texte est stylisé avec styled-jsx.</p>

      <style jsx>{`
        h1 {
          color: blue;
          font-size: 24px;
        }
        p {
          color: green;
          font-size: 16px;
        }
      `}</style>
    </div>
  );
}
```

FIGURE 2.34 – Exemple de styles

Chapitre 3 : Styling avec TailwindCSS

Chapitre 3

Styling avec TailwindCSS

I Configuration de TailwindCSS

Pour commencer à utiliser TailwindCSS, vous devez d'abord l'installer et le configurer dans votre projet. TailwindCSS est un framework CSS utilitaire qui vous permet de styliser vos applications rapidement.

— **Installation** : Vous pouvez installer TailwindCSS via npm ou yarn. Utilisez la commande suivante pour l'installer :

```
npm install tailwindcss
```

— **Configuration** : Après l'installation, vous devez créer un fichier de configuration en exécutant :

```
npx tailwindcss init
```

Cela génère un fichier `tailwind.config.js` où vous pouvez personnaliser votre configuration.

II Bases en TailwindCSS

1 Couleurs et arrière-plans

TailwindCSS offre une large palette de couleurs que vous pouvez utiliser pour styliser vos éléments. Vous pouvez appliquer des couleurs de texte et des arrière-plans en utilisant des classes utilitaires.

— **Couleurs de texte** : Utilisez des classes comme `text-blue-500` pour définir la couleur du texte.

— **Arrière-plans** : Utilisez des classes comme `bg-gray-200` pour définir la couleur de l'arrière-plan.

Exemple :

```
<div class="text-blue-500 bg-gray-200">
  Ceci est un texte bleu sur fond gris.
</div>
```

2 Tailles et positionnements

TailwindCSS fournit des classes pour contrôler les tailles et les positionnements des éléments.

— **Tailles** : Utilisez des classes comme `w-1/2` pour définir la largeur et `h-64` pour définir la hauteur.

— **Positionnements** : Utilisez des classes comme `relative`, `absolute`, `top-0`, et `left-0` pour positionner les éléments.

Exemple :

```
<div class="w-1/2 h-64 relative">
  <div class="absolute top-0 left-0">
    Ceci est un élément positionné.
  </div>
</div>
```

3 Texts et fonts

TailwindCSS permet de styliser le texte et les polices facilement.

— **Tailles de texte** : Utilisez des classes comme `text-lg` pour définir la taille du texte.

— **Polices** : Utilisez des classes comme `font-bold` pour définir le poids de la police.

Exemple :

```
<p class="text-lg font-bold">
  Ceci est un texte en gras et de grande taille.
</p>
```

III Mises en page TailwindCSS

1 Flexing

TailwindCSS offre des classes pour utiliser Flexbox, ce qui facilite la création de mises en page flexibles.

— **Flex Container** : Utilisez la classe `flex` pour créer un conteneur flex.

— **Direction** : Utilisez des classes comme `flex-row` ou `flex-col` pour définir la direction des éléments flex.

Exemple :

```
<div class="flex flex-row">
  <div>Élément 1</div>
  <div>Élément 2</div>
</div>
```

2 Griding

TailwindCSS prend également en charge CSS Grid pour des mises en page plus complexes.

— **Grid Container** : Utilisez la classe `grid` pour créer un conteneur de grille.

— **Columns** : Utilisez des classes comme `grid-cols-3` pour définir le nombre de colonnes.

Exemple :

```
<div class="grid grid-cols-3 gap-4">
  <div>Élément 1</div>
  <div>Élément 2</div>
  <div>Élément 3</div>
</div>
```

IV Pseudo-classes et group-hover

TailwindCSS permet de styliser les éléments en utilisant des pseudo-classes comme `hover`, `focus`, etc.

— **Hover** : Utilisez des classes comme `hover:bg-gray-300` pour changer le style au survol.

— **Group Hover** : Utilisez des classes comme `group-hover:text-blue-500` pour appliquer des styles de groupe.

Exemple :

```
<div class="group hover:bg-gray-300">
  <p class="group-hover:text-blue-500">
    Passez la souris sur moi.
  </p>
</div>
```

V Animations

TailwindCSS offre des classes pour ajouter des animations à vos éléments.

— **Animations** : Utilisez des classes comme `animate-spin` pour appliquer des animations prédéfinies.

Exemple :

```
<div class="animate-spin">
  Ceci est un élément en rotation.
</div>
```

VI Contrôle de responsivité

TailwindCSS facilite la création de designs responsifs avec des classes spécifiques pour différentes tailles d'écran.

— **Préfixes de taille d'écran** : Utilisez des préfixes comme `md:`, `lg:`, etc., pour appliquer des styles spécifiques à certaines tailles d'écran.

Exemple :

```
<div class="text-sm md:text-lg lg:text-xl">
  Ceci est un texte responsif.
</div>
```

Chapitre 4 : Front-End en Next.js

Chapitre 4

Front-End en Next.js

I Architecture de Next.js

Next.js est un framework React qui permet de créer des applications web avec rendu côté serveur (SSR) et génération de sites statiques (SSG). Il offre une excellente expérience développeur avec des fonctionnalités comme le routage automatique, l'optimisation des images, et bien plus encore.

II Routage

Le routage dans Next.js est basé sur le système de fichiers. Chaque fichier dans le dossier `pages` est automatiquement associé à une route.

1 Fichier principale

Le fichier **_app.jsx** est le point d'entrée principal de votre application Next.js. Il initialise l'application et peut inclure des configurations globales.

Exemple :

```
1  export default function MyApp({ Component, pageProps }) {
2    return <Component {...pageProps} />;
3  }
4
```

FIGURE 4.1 – $_appjsx$

2 Le fichier `_document.jsx` dans Next.js

Le fichier **_document.jsx** est un fichier spécial dans Next.js qui permet de personnaliser la structure HTML globale de l'application. Contrairement aux autres composants, il ne s'exécute que côté serveur lors du rendu initial.

— **But principal :** Modifier l'élément **<html>**, **<head>** ou **<body>** du document.

— **Utilisations courantes :**
  — Ajouter des balises de polices (Google Fonts).
  — Ajouter un favicon.
  — Définir la langue du document avec **<Html lang="...">**.
  — Ajouter des classes personnalisées à **<body>** pour des thèmes (ex. dark mode).
— **Important :** Aucun hook React ou logique interactive ne doit être utilisé ici.

```jsx
import { Html, Head, Main, NextScript } from 'next/document';

export default function Document() {
  return (
    <Html lang="fr">
      <Head>
        <link rel="icon" href="/favicon.ico" />
        <link href="https://fonts.googleapis.com/css2?family=Roboto" rel="stylesheet" />
      </Head>
      <body>
        <Main /> // Toutes les routes dans le dossier pages seront injectées ici automatiquement
        <NextScript /> // Tous les scripts seront injectés également ici
      </body>
    </Html>
  );
}
```

FIGURE 4.2 – $_d ocument jsx$

Exemple simple :

Différence avec `_app.jsx` :

— `_app.jsx` : personnalise le comportement global de l'application (layout, context, styles globaux). Exécuté côté client et serveur.
— `_document.jsx` : personnalise uniquement la structure HTML du document. Exécuté uniquement côté serveur.

3 Hook de routage : useRouter

Next.js fournit un hook pour gérer le routage dans l'application. Le hook **useRouter** permet d'accéder à l'objet **router** pour naviguer entre les pages et accéder aux paramètres de l'URL.

3.1 Navigation

Le hook **useRouter** offre plusieurs méthodes pour naviguer entre les pages.

Push : La méthode **push** permet de naviguer vers une nouvelle page en ajoutant une nouvelle entrée dans l'historique du navigateur.

Exemple :

```
1   import { useRouter } from 'next/router';
2
3   function MyComponent() {
4     const router = useRouter();
5
6     const handleClick = () => {
7       router.push('/about');
8     };
9
10    return <button onClick={handleClick}>Go to About</button>;
11  }
12
```

FIGURE 4.3 – useRouter

Replace La méthode **replace** remplace l'entrée actuelle dans l'historique du navigateur par une nouvelle URL.

Exemple :

```
1   import { useRouter } from 'next/router';
2
3   function MyComponent() {
4     const router = useRouter();
5
6     const handleClick = () => {
7       router.replace('/contact');
8     };
9
10    return <button onClick={handleClick}>Go to Contact</button>;
11  }
12
```

FIGURE 4.4 – useRouter-replace

Reload La méthode `reload` recharge la page actuelle.

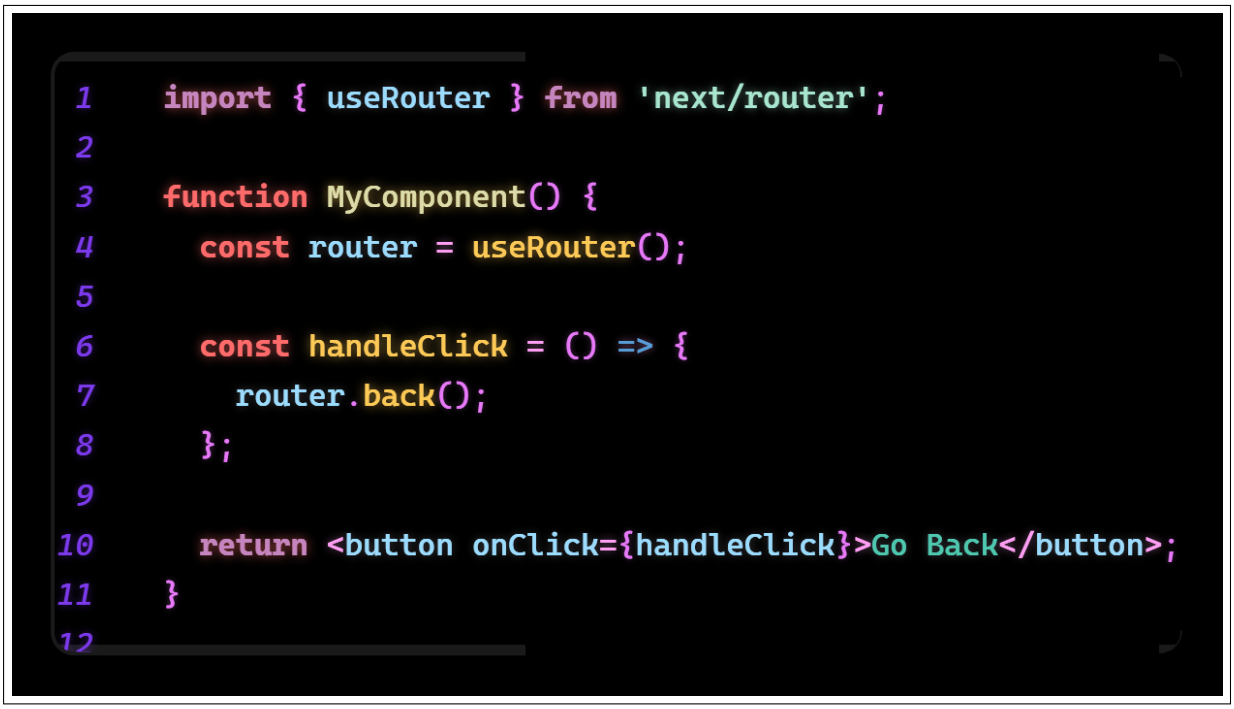
Exemple :

```
import { useRouter } from 'next/router';

function MyComponent() {
  const router = useRouter();

  const handleClick = () => {
    router.reload();
  };

  return <button onClick={handleClick}>Reload Page</button>;
}
```

FIGURE 4.5 – useRouter-reload

Back La méthode `back` permet de revenir à la page précédente dans l'historique du navigateur.

Exemple :



```
import { useRouter } from 'next/router';

function MyComponent() {
  const router = useRouter();

  const handleClick = () => {
    router.back();
  };

  return <button onClick={handleClick}>Go Back</button>;
}
```

FIGURE 4.6 – useRouter-back

3.2 Pathname et AsPath

`useRouter` fournit des informations sur le chemin actuel de l'URL.

— `pathname` : Le chemin de l'URL actuelle sans les paramètres de requête.

— `asPath` : Le chemin complet de l'URL, y compris les paramètres de requête.

Exemple : imaginons si on est sur **www.webologie.com/Profile ?userID=123123**

```
1   import { useRouter } from 'next/router';
2
3   function MyComponent() {
4     const router = useRouter();
5
6     return (
7       <div>
8         <p>Pathname: "{router.pathname}"</p> // Pathname: "/Profile"
9         <p>AsPath: "{router.asPath}"</p> // AsPath: "/Profile?userID=123123"
10      </div>
11    );
12  }
13
```

FIGURE 4.7 – useRouter-pathname-aspath

3.3 Router query

Le hook `useRouter` permet également d'accéder aux paramètres de requête de l'URL.
Exemple : imaginons si on est sur **www.webologie.com/Profile ?userID=123123**

```
1   import { useRouter } from 'next/router';
2
3   function MyComponent() {
4     const router = useRouter();
5     const { id } = router.query; // C'est obligatoire d'avoir le même nom
6     const { userID } = router.query; // Comme ça c'est correct
7
8     return <div>User ID: {id}</div>; // User ID: undefined
9     return <div>User ID: {userID}</div>; // User ID: 123123
10  }
11
```

FIGURE 4.8 – useRouter-query

3.4 Différence entre useRouter et window

Alors que **useRouter** est spécifique à Next.js et offre des fonctionnalités avancées pour le routage dans une application Next.js, l'objet **window** est une API globale du navigateur qui permet de manipuler l'historique de navigation et d'autres fonctionnalités du navigateur.

Exemple de différence :

— **useRouter** est intégré à Next.js et permet une navigation côté client optimisée.

— **window** est une API standard du navigateur et peut être utilisée dans n'importe quelle application web.

4 Routes

Next.js fournit plusieurs composants pour gérer les routes dans votre application.

4.1 Link

Le composant **Link** permet de créer des liens entre les pages sans recharger la page.
Exemple de Link :

```
1   import Link from 'next/link';
2
3   function Home() {
4     return (
5       <Link href="/About">
6         <p>About Us</p>
7       </Link>
8     );
9   }
10
```

FIGURE 4.9 – Exemple de Link

Remarque : Ne placez jamais **<a>** à l'interieur de **<Link>**, ça va causer une erreur de Hydration !

FIGURE 4.10 – Link erreur

4.2 Head

Le composant `Head` permet de modifier les balises `head` du document HTML, comme le titre et les méta-données pour le SEO (Search Engine Optimization).

Exemple de Head :

```
import Head from 'next/head';

function MyPage() {
  return (
    <div>
      <Head>
        <title>My Page Title</title>
        <meta name="description" content="This is my page description" />
        <link rel="icon" href="/logo.png" />
      </Head>
      <h1>Welcome to My Page</h1>
    </div>
  );
}
```

FIGURE 4.11 – Exemple de Head

Chapitre 5 : Backend-End en Next.js

Chapitre 5

Backend-End en Next.js

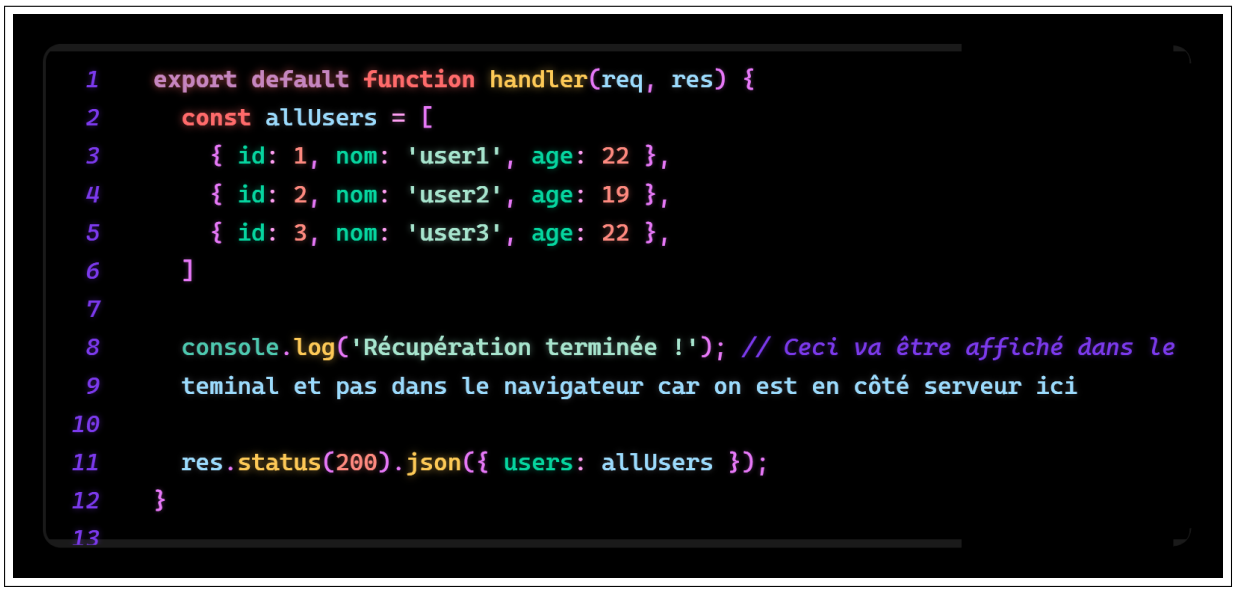
I Conception des APIs

1 Emplacement et contexte

Les APIs dans Next.js sont généralement placées dans le dossier **pages/api**. Chaque fichier dans ce dossier est traité comme un endpoint API. Par exemple, un fichier **pages/api/getUsers.js** sera accessible via l'URL **/api/getUsers**.

2 Fonction handler et réponses serveur

Dans Next.js, chaque API route doit exporter une fonction par défaut qui prend deux arguments : **req** (l'objet de requête) et **res** (l'objet de réponse).



```
export default function handler(req, res) {
  const allUsers = [
    { id: 1, nom: 'user1', age: 22 },
    { id: 2, nom: 'user2', age: 19 },
    { id: 3, nom: 'user3', age: 22 },
  ]

  console.log('Récupération terminée !'); // Ceci va être affiché dans le
  teminal et pas dans le navigateur car on est en côté serveur ici

  res.status(200).json({ users: allUsers });
}
```

FIGURE 5.1 – getUsers

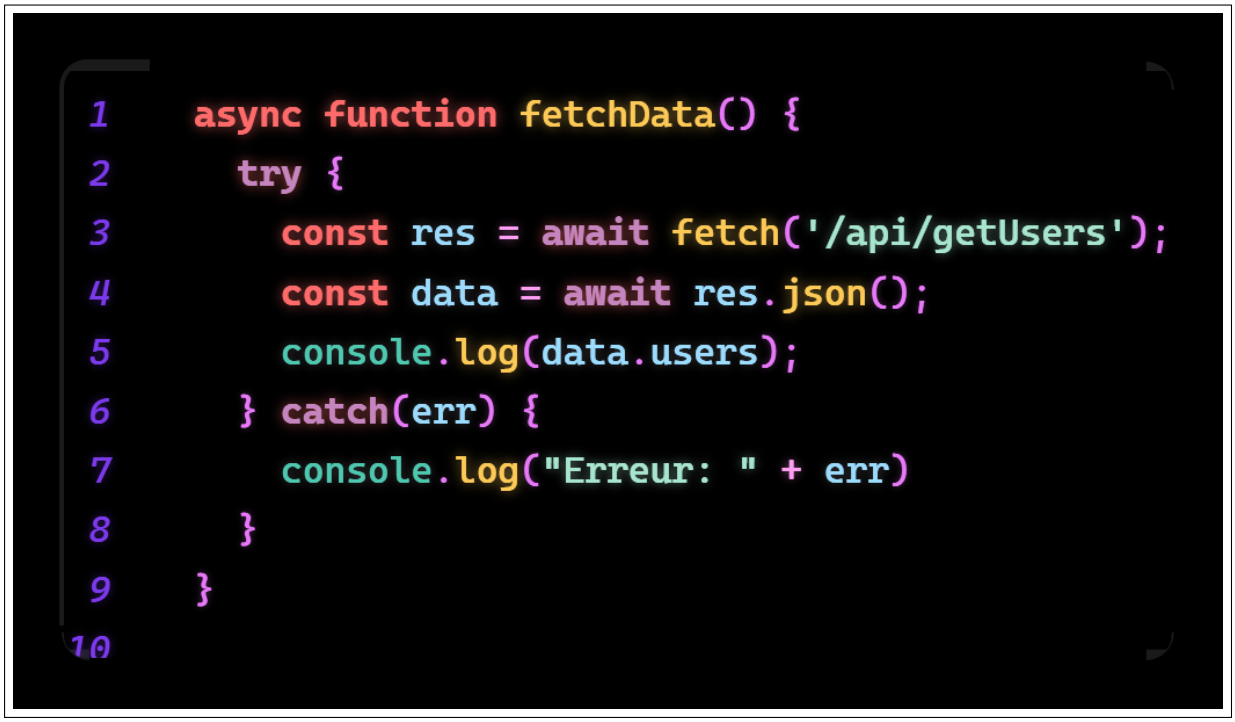
— **req** : Représente la requête HTTP et contient des propriétés pour les paramètres de requête (body, query, method, cookies, cookies...).

— **res** : Utilisé pour envoyer des réponses au client. Vous pouvez définir le statut HTTP et envoyer des données, généralement au format JSON.

3 Fetching et récupération de données

3.1 Envoi d'une requête HTTP avec fetch

Pour récupérer des données depuis une API externe dans Next.js, vous pouvez utiliser la fonction **fetch** dans votre code côté serveur ou côté client.



```
async function fetchData() {
  try {
    const res = await fetch('/api/getUsers');
    const data = await res.json();
    console.log(data.users);
  } catch(err) {
    console.log("Erreur: " + err)
  }
}

```

FIGURE 5.2 – Envoi d'une requête HTTP avec fetch

— **fetch** : Une API JavaScript pour récupérer des ressources à travers le réseau de manière asynchrone.

3.2 Paramètres d'une requête HTTP

Pour récupérer des données en faisant passer un argument spécifique on peut utiliser cette méthode :

```
async function fetchData() {
  try {
    const res = await fetch(`/api/getUsers?minAge=22`);
    const data = await res.json();
    console.log(data.users);
  } catch(err) {
    console.log("Erreur: " + err)
  }
}

```

FIGURE 5.3 – Paramètres d'une requête HTTP frontend

Front-End :

```
export default function handler(req, res) {
  const minAge = req.query; // Attention ! "query" et pas "body"

  const allUsers = [
    { id: 1, nom: 'user1', age: 22 },
    { id: 2, nom: 'user2', age: 19 },
    { id: 3, nom: 'user3', age: 22 },
  ]

  const usersWithAge22 = allUsers.filter(user => user.age >= minAge);

  res.status(200).json({ users: usersWithAge22 });
}

```

FIGURE 5.4 – Paramètres d'une requête HTTP backend

Back-End :

3.3 Corps d'une requête HTTP

Pour récupérer des données plus précisement, on peut utiliser des headers, methode, et body dans un appel API :

```
async function fetchData() {
  try {
    const response = await fetch("/api/getUsers",
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ minAge: 22, nom: 'user1' }),
    });

    const data = await response.json();
    console.log(data.users);
  } catch(err) {
    console.error("Erreur: " + err);
  }
}
```

FIGURE 5.5 – Corps d'une requête HTTP

Front-End :

```
export default function handler(req, res) {
  if(req.method !== 'POST') {
    return res.status(405).json("Méthode non autorisée");
  }
  const { nom, minAge } = req.body; // Attention ! "body" et pas "query"

  const allUsers = [
    { id: 1, nom: 'user1', age: 22 },
    { id: 2, nom: 'user2', age: 19 },
    { id: 3, nom: 'user3', age: 22 },
  ]

  const usersWithAge22 = allUsers.filter(user => user.age >= minAge && user.nom === nom);

  res.status(200).json({ users: usersWithAge22 });
}
```

FIGURE 5.6 – Corps d'une requête HTTP backend

Back-End :

3.4 Méthodes d'une requête HTTP

Les méthodes HTTP définissent le type d'action à effectuer sur une ressource. Voici quelques-unes des méthodes les plus couramment utilisées :

- **GET** : Utilisée pour demander des données à un serveur. C'est la méthode la plus courante pour récupérer des informations.
- **POST** : Utilisée pour envoyer des données à un serveur afin de créer ou de mettre à jour une ressource.
- **PUT** : Utilisée pour envoyer des données à un serveur afin de créer ou remplacer une ressource.
- **DELETE** : Utilisée pour supprimer une ressource spécifiée.
- **PATCH** : Utilisée pour appliquer des modifications partielles à une ressource.

Exemple d'utilisation des méthodes HTTP avec fetch :

```
async function fetchData() {
  // GET request
  const getResponse = await fetch('/api/data', {
    method: 'GET'
  });

  // POST request
  const postResponse = await fetch('/api/data', {
    method: 'POST',
    body: JSON.stringify({ key: 'value' }),
    headers: {
      'Content-Type': 'application/json'
    }
  });
}
```

FIGURE 5.7 – Exemple d'utilisation des méthodes HTTP avec fetch

3.5 Headers d'une requête HTTP

Les headers HTTP permettent au client et au serveur de passer des informations supplémentaires avec une requête ou une réponse. Voici quelques headers courants :

— **Content-Type** : Indique le type de média de la ressource. Par exemple, `application/json` pour du JSON.

— **Authorization** : Contient les informations d'authentification du client.

— **Accept** : Informe le serveur des types de médias acceptables pour la réponse.

— **User-Agent** : Contient des informations sur l'application cliente qui fait la requête.

Exemple d'utilisation des headers avec fetch :

```
async function fetchDataWithHeaders() {
  const response = await fetch('/api/data', {
    method: 'GET',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer your_token_here'
    }
  });
  const data = await response.json();
  console.log(data);
}
```

FIGURE 5.8 – Exemple d'utilisation des headers avec fetch

3.6 Utilisation de axios

Axios est une bibliothèque populaire pour effectuer des requêtes HTTP. Elle fonctionne à la fois dans le navigateur et dans Node.js, ce qui la rend très utile pour les applications Next.js.

```
import axios from 'axios';

async function fetchData() {
  try {
    const response = await axios.post('/api/getUsers', { minAge: 22, nom: 'user1' });
    console.log(response.data);
  } catch (err) {
    console.error("Erreur: " + err);
  }
}
```

FIGURE 5.9 – Utilisation de axios

— **Axios** : Une bibliothèque JavaScript qui permet de faire des requêtes HTTP depuis le navigateur ou Node.js. Elle supporte les Promesses et permet d'intercepter les requêtes et les réponses.

4 Traitement de la réponse du serveur

4.1 Les attributs de l'objet renvoyé par l'API

Lorsqu'on fetch sur une API (**const res = await fetch("/api/getUsers")**, elle revoie un objet qui inclut les informations retournées par le serveur :

4.2 Traitement par ok / not ok

On peut vérifier si la réponse est réussie en utilisant la propriété **ok** de l'objet réponse :

```
const res = await fetch('/api/getUsers');
if (res.ok) {
  const data = await res.json();
  console.log('Data fetched successfully:', data);
} else {
  console.error('Failed to fetch data');
}
```

FIGURE 5.10 – Traitement par ok / not ok

Attribut / Méthode	Type	Description
`res.ok`	Bool	Vaut `true` si le code de statut est entre 200 et 299
`res.status`	Int	Code de réponse HTTP (ex : 200, 404, 500)
`res.statusText`	String	Texte associé au code HTTP (ex : `"OK"`, `"Not Found"`)
`res.headers`	Object Headers	Liste des en-têtes de la réponse (accès via `get()`)
`res.url`	String	URL de la réponse
`res.type`	String	Type de réponse : `"basic"`, `"cors"`, `"opaque"`
`res.redirected`	Bool	Vaut `true` si une redirection a eu lieu
`res.body`	ReadableStream	Flux de données de la réponse brute
`res.json()`	Promise	Lit la réponse et la convertit en objet JSON
`res.text()`	Promise	Lit la réponse comme texte brut
`res.blob()`	Promise	Lit la réponse comme fichier binaire
`res.formData()`	Promise	Lit la réponse comme données de formulaire

TABLEAU 5.1 : Attributs et méthodes de l'objet `Response` dans Fetch API

4.3 Traitement par code de réponse

Il est également possible de traiter les réponses en fonction de leur code de statut HTTP :

```
const res = await fetch('/api/getUsers');
if (res.status === 200) {
  const data = await res.json();
  console.log('Data:', data);
} else if (res.status === 404) {
  console.error('Data not found');
} else {
  console.error('An error occurred');
}
```

Code	Nom	Description
200	OK	La requête a réussi. Réponse standard pour une requête HTTP réussie.
201	Created	La requête a abouti et une ressource a été créée. Utilisé généralement après une requête POST.
204	No Content	La requête a réussi, mais il n'y a pas de contenu à renvoyer.
301	Moved Permanently	La ressource a été déplacée de façon permanente à une nouvelle URL.
302	Found	Redirection temporaire vers une autre URL.
304	Not Modified	La ressource n'a pas été modifiée depuis la dernière requête. Utilisé avec le cache.
400	Bad Request	La requête est mal formée ou invalide.
401	Unauthorized	L'utilisateur n'est pas authentifié ou ne fournit pas de jeton valide.
403	Forbidden	L'utilisateur est authentifié mais n'a pas le droit d'accéder à la ressource.
404	Not Found	La ressource demandée est introuvable.
405	Method Not Allowed	La méthode HTTP utilisée (GET, POST...) n'est pas autorisée pour cette ressource.
500	Internal Server Error	Erreur générique du serveur (bug, crash, etc.).
502	Bad Gateway	Mauvaise réponse envoyée par un serveur intermédiaire (ex. proxy).
503	Service Unavailable	Le serveur est indisponible, souvent pour cause de maintenance.
504	Gateway Timeout	Le serveur intermédiaire a expiré en attendant une réponse.

TABLEAU 5.2 : Les codes de statut HTTP les plus utilisés

5 Variables d'environnement (dotenv)

5.1 Définition

Next.js prend en charge les variables d'environnement via l'utilisation d'un fichier **.env.local** ou bien **.env**. Vous pouvez y stocker des informations sensibles comme les clés API.

FIGURE 5.11 – Exemple env

Si on veut héberger notre siteweb ou le mettre sur **GitHub**, il faut s'assurer d'ajouter **.env** à votre fichier **.gitignore** pour éviter de commiter des informations sensibles.

5.2 Récupération des variables d'environement

— **dotenv** : Est un module qui charge les variables d'environnement à partir d'un fichier `.env` dans **process.env**.

— **process.env** : Est accessible seulement en côté serveur, sinon on doit leur ajouter le suffix **NEXT_PUBLIC**

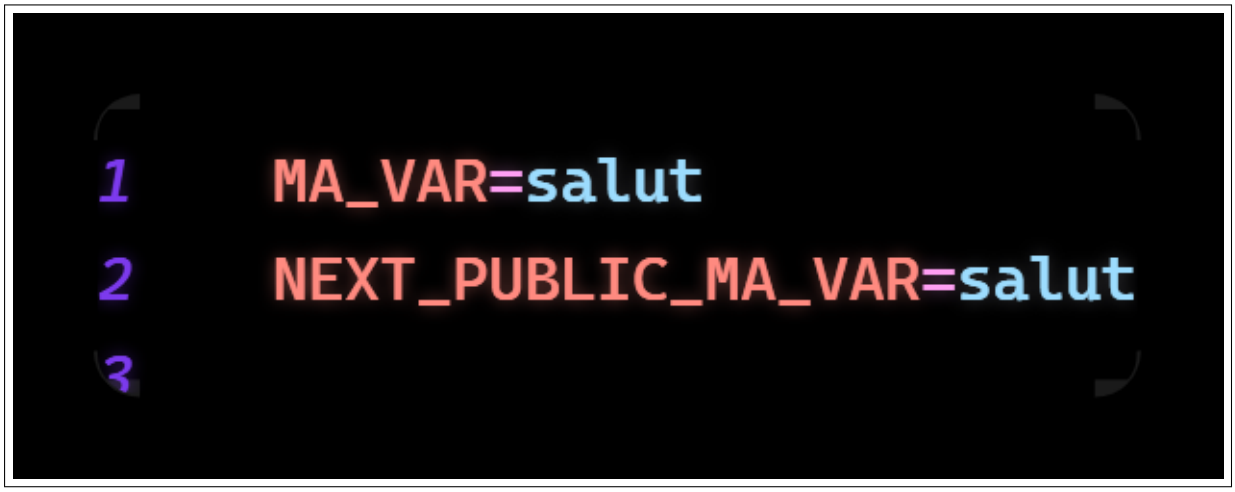


FIGURE 5.12 – Exemple de variable dans env

Exemple de variable dans .env :

```
console.log(process.env.MA_VAR); // affiche undefined
console.log(process.env.NEXT_PUBLIC_MA_VAR); // affiche "salut"
```

FIGURE 5.13 – Exemple de chargement de ces variables côté client

Exemple de chargement de ces variables côté client :

```
console.log(process.env.MA_VAR); // affiche "salut"
console.log(process.env.NEXT_PUBLIC_MA_VAR); // affiche "salut"
```

FIGURE 5.14 – Exemple de chargement de ces variables côté serveur

Exemple de chargement de ces variables côté serveur :

II MySQL

1 Connexion à la base de données

Pour se connecter à une base de données MySQL dans une application Next.js, vous pouvez utiliser le module ‘mysql2‘ qui est une bibliothèque populaire pour interagir avec MySQL en Node.js.

Voici un exemple de code pour établir une connexion à une base de données MySQL **(/api/_lib/connect.js)** :

```
import mysql from 'mysql2/promise';

export default pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost', // Adresse du serveur MySQL
  user: process.env.DB_USER || 'user', // Nom d'utilisateur MySQL
  password: process.env.DB_PASS || 'password', // Mot de passe MySQL
  database: process.env.DB_NAME || 'database_name', // Nom de la base de données
  connectionLimit: 10, // Nombre max de connexions simultanées ouvertes dans le pool.
  waitForConnections: true, // Les requêtes suivantes attendront qu'une connexion se libère au lieu de générer une erreur immédiatement.
  queueLimit: 0 // Nombre max de requêtes en file d'attente.
});

```

FIGURE 5.15 – Connexion MySQL

— **mysql2** : Une bibliothèque Node.js pour interagir avec MySQL. Elle prend en charge les promesses, ce qui facilite l’écriture de code asynchrone.

— **Pool de connexions** : Un pool de connexions est utilisé pour gérer et réutiliser les connexions à la base de données, ce qui améliore les performances de l’application.

2 Traitement des requêtes

Une fois la connexion établie, vous pouvez exécuter des requêtes SQL pour interagir avec votre base de données. Voici quelques exemples de traitement de requêtes courantes :

2.1 Exemple d'insertion de données

Pour insérer des données dans une table, vous pouvez utiliser la requête SQL 'INSERT INTO'.

```
1   import db from '/api/_lib/connect';
2
3   async function addUser(name, email) {
4     const [result] = await db.execute(
5       'INSERT INTO users (name, email) VALUES (?, ?)',
6       [name, email]
7     );
8     return result.insertId;
9   }
10
```

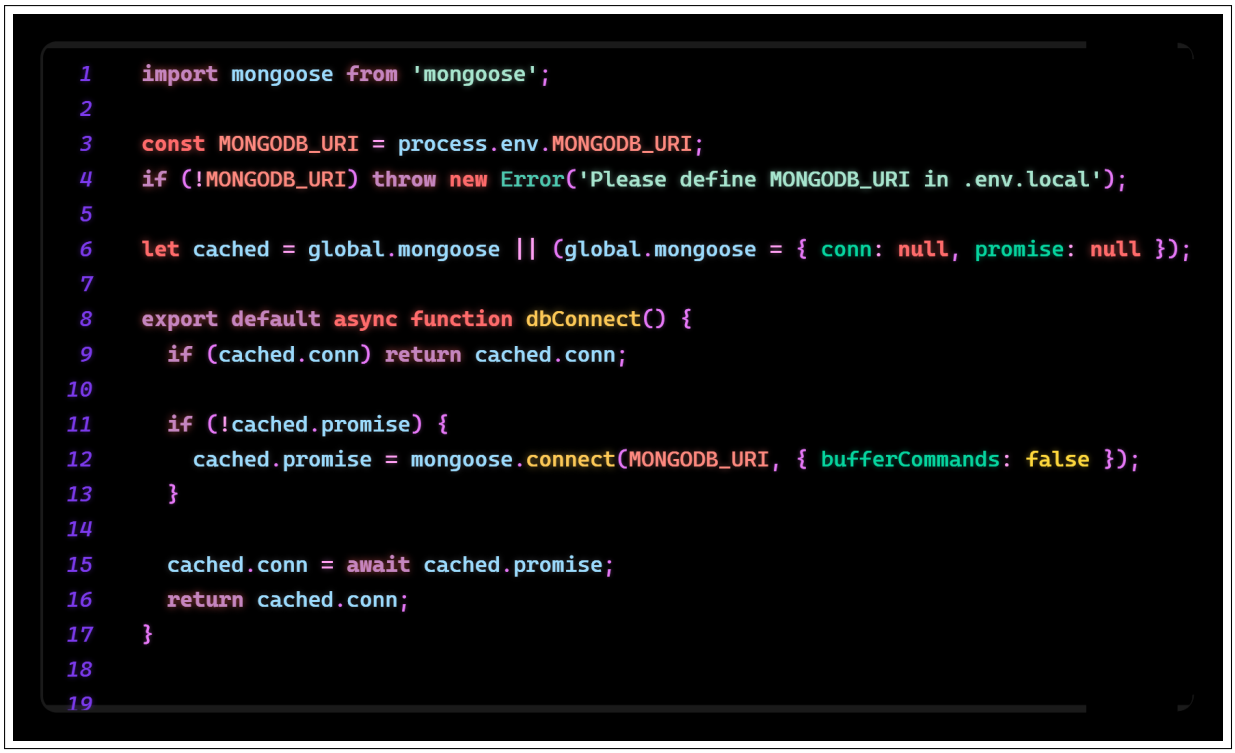
FIGURE 5.16 – Insertion MySQL

III MongoDB

1 Connexion à la base de données

Pour se connecter à une base de données MongoDB dans une application Next.js, vous pouvez utiliser le module 'mongoose', qui est une bibliothèque de modélisation d'objets MongoDB conçue pour fonctionner dans un environnement asynchrone.

Voici un exemple de code pour établir une connexion à une base de données MongoDB **(/api/_lib/connect.js)** :



```
import mongoose from 'mongoose';

const MONGODB_URI = process.env.MONGODB_URI;
if (!MONGODB_URI) throw new Error('Please define MONGODB_URI in .env.local');

let cached = global.mongoose || (global.mongoose = { conn: null, promise: null });

export default async function dbConnect() {
  if (cached.conn) return cached.conn;

  if (!cached.promise) {
    cached.promise = mongoose.connect(MONGODB_URI, { bufferCommands: false });
  }

  cached.conn = await cached.promise;
  return cached.conn;
}
```

FIGURE 5.17 – Connexion MongoDB

— **Mongoose** : Une bibliothèque pour Node.js qui permet de définir des schémas avec une typage fort pour vos données MongoDB.

— **MONGODB_URI** : L'URI de connexion à votre base de données MongoDB, généralement stockée dans une variable d'environnement.

2 Modèles

Les modèles Mongoose sont utilisés pour interagir avec les collections MongoDB. Ils définissent la structure des documents stockés dans ces collections. On va utiliser **/api/_models/**

2.1 Types normales

Les types normaux dans Mongoose incluent des types de données comme String, Number, Date, Boolean, etc.

```javascript
import mongoose from 'mongoose';

const UserSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  age: {
    type: Number
  }
});

export default mongoose.models.User || mongoose.model('User', UserSchema);
```

FIGURE 5.18 – Modèle normal MongoDB

2.2 Types pointeurs

Les types pointeurs, ou références, sont utilisés pour créer des relations entre différents modèles.

```javascript
const PostSchema = new mongoose.Schema({
  title: String,
  body: String,
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  }
});

export default mongoose.models.Post || mongoose.model('Post', PostSchema);
```

FIGURE 5.19 – Modèle pointeur MongoDB

3 Opérations CRUD et Population

3.1 Exemple de sélection multiple

Pour sélectionner des documents dans une collection, vous pouvez utiliser la méthode ‘find‘.

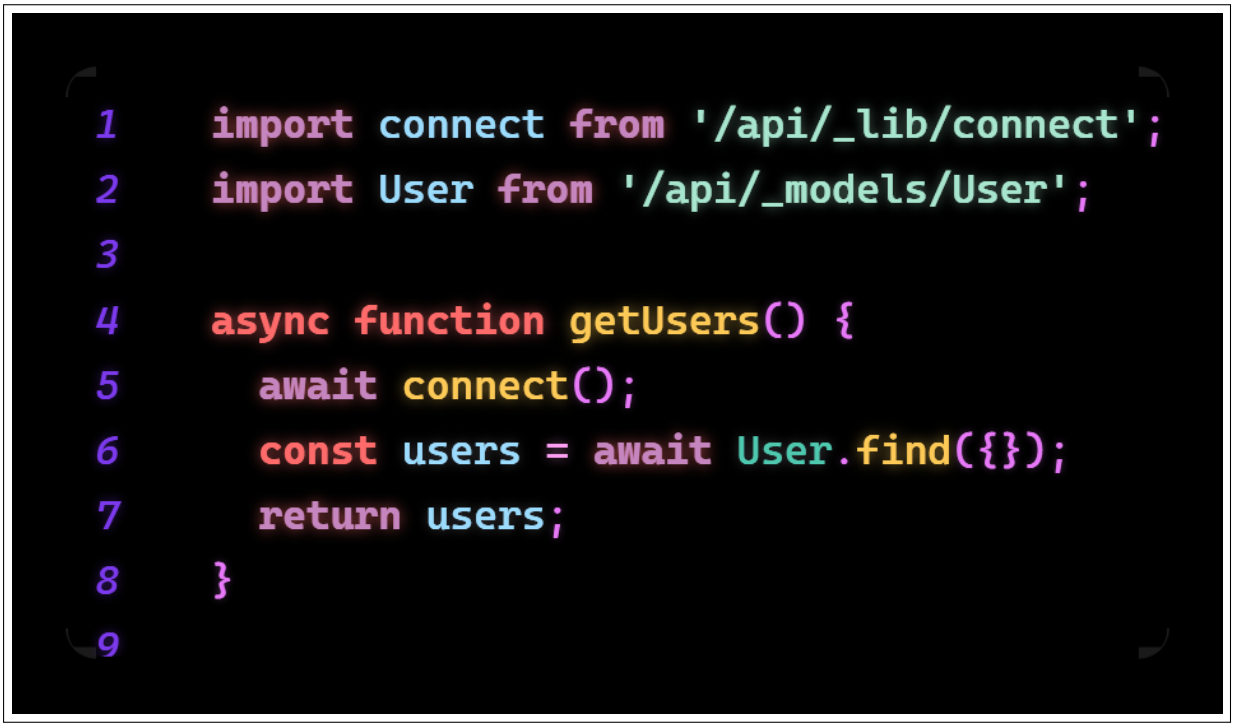


FIGURE 5.20 – Exemple de sélection multiple MongoDB

3.2 Lecture / Sélection

- **find(filter, options)** : Renvoie un curseur sur les documents correspondant au filtre.
- **findOne(filter, options)** : Renvoie un seul document correspondant.
- **findOneAndDelete(filter, options)** : Supprime un document et renvoie sa valeur.
- **findOneAndUpdate(filter, update, options)** : Met à jour un document et renvoie la version avant ou après.
- **findOneAndReplace(filter, replacement, options)** : Remplace un document entier.

3.3 Insertion

- **insertOne(document, options)** : Insère un seul document.
- **insertMany(documents, options)** : Insère plusieurs documents.

3.4 Mise à jour

- **updateOne(filter, update, options)** : Met à jour un seul document.
- **updateMany(filter, update, options)** : Met à jour plusieurs documents.
- **replaceOne(filter, replacement, options)** : Remplace un document correspondant.

3.5 Suppression

- **deleteOne(filter, options)** : Supprime un seul document.
- **deleteMany(filter, options)** : Supprime plusieurs documents.
- **bulkWrite(operations, options)** : Exécute plusieurs opérations en une seule commande.

3.6 Comptage

- **countDocuments(filter, options)** : Compte les documents correspondant au filtre.
- **estimatedDocumentCount()** : Renvoie une estimation du nombre total de documents.

3.7 Gestion de collection

- **drop()** : Supprime la collection.
- **rename(newName, options)** : Renomme la collection.

3.8 Population

La population est un mécanisme proposé par Mongoose qui permet de remplir automatiquement les champs faisant référence à des documents d'autres collections, en remplaçant les **_id** par les documents complets liés.

Principe : Lorsqu'un document contient une référence à un autre via un identifiant (par exemple **userId**), la population permet de récupérer et d'intégrer automatiquement les données de cet utilisateur dans le résultat final.

```
1   const postSchema = new mongoose.Schema({
2     title: String,
3     user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
4   });
5
```

FIGURE 5.21 – Exemple simple d'un modèle Post avec référence à User

Exemple simple d'un modèle Post avec référence à User :

Sans population → **Post.find()** renvoie :

FIGURE 5.22 – Résultat sans population

Avec population → **Post.find().populate('user')** renvoie :

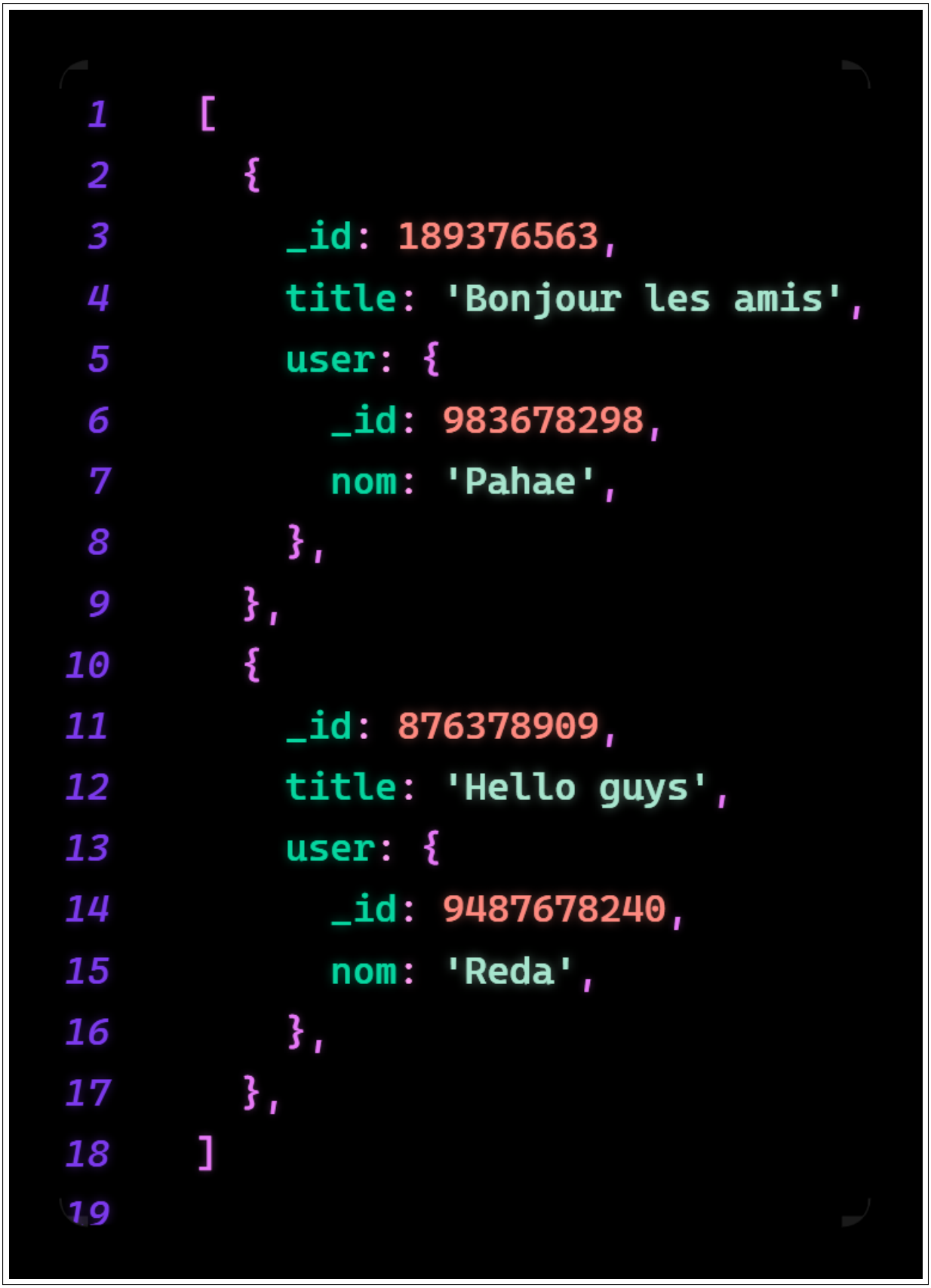


FIGURE 5.23 – Résultat avec population

```
Post.find()
  .populate({ path: 'user', select: 'nom email' }) // Uniquement nom et email
  .exec();

```

FIGURE 5.24 – Sélection de champs spécifiques

Sélection de champs spécifiques :

```
Post.find()
  .populate({ path: 'user', select: '-password' }) // Exclure password
  .exec();

```

FIGURE 5.25 – Exclusion de champs spécifiques

Exclusion de champs spécifiques :

IV Gestion des fichiers

1 Définition d'un BLOB

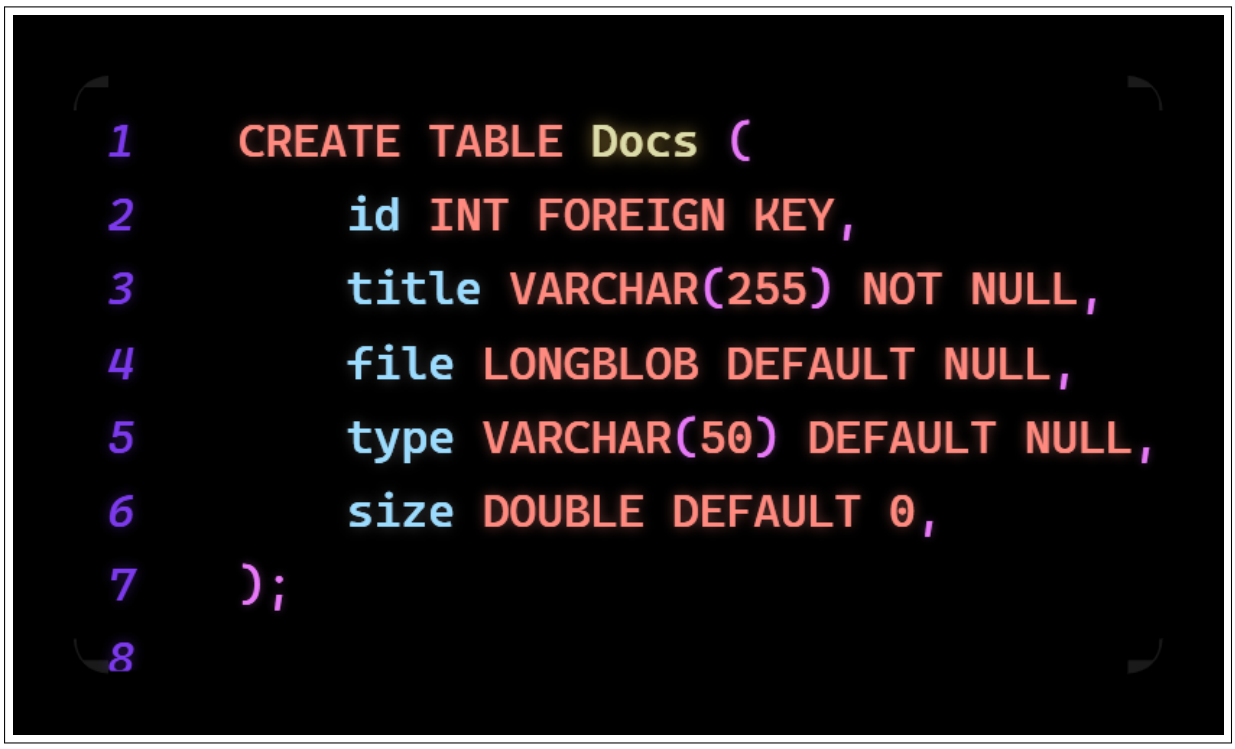
Un BLOB est un type de donnée utilisé dans les bases de données pour stocker des données binaires de grande taille, comme :

- images
- audios
- vidéos
- fichiers PDF, Word, etc.

Type	Taille maximale
TINYBLOB	255 octets
BLOB	65 535 octets (~64 Ko)
MEDIUMBLOB	16 777 215 octets (~16 Mo)
LONGBLOB	4 294 967 295 octets (~4 Go)

TABLEAU 5.3 : Types de BLOB en MySQL et leurs tailles maximales

2 Traitement BDD



```
1    CREATE TABLE Docs (
2        id INT FOREIGN KEY,
3        title VARCHAR(255) NOT NULL,
4        file LONGBLOB DEFAULT NULL,
5        type VARCHAR(50) DEFAULT NULL,
6        size DOUBLE DEFAULT 0,
7    );
8
```

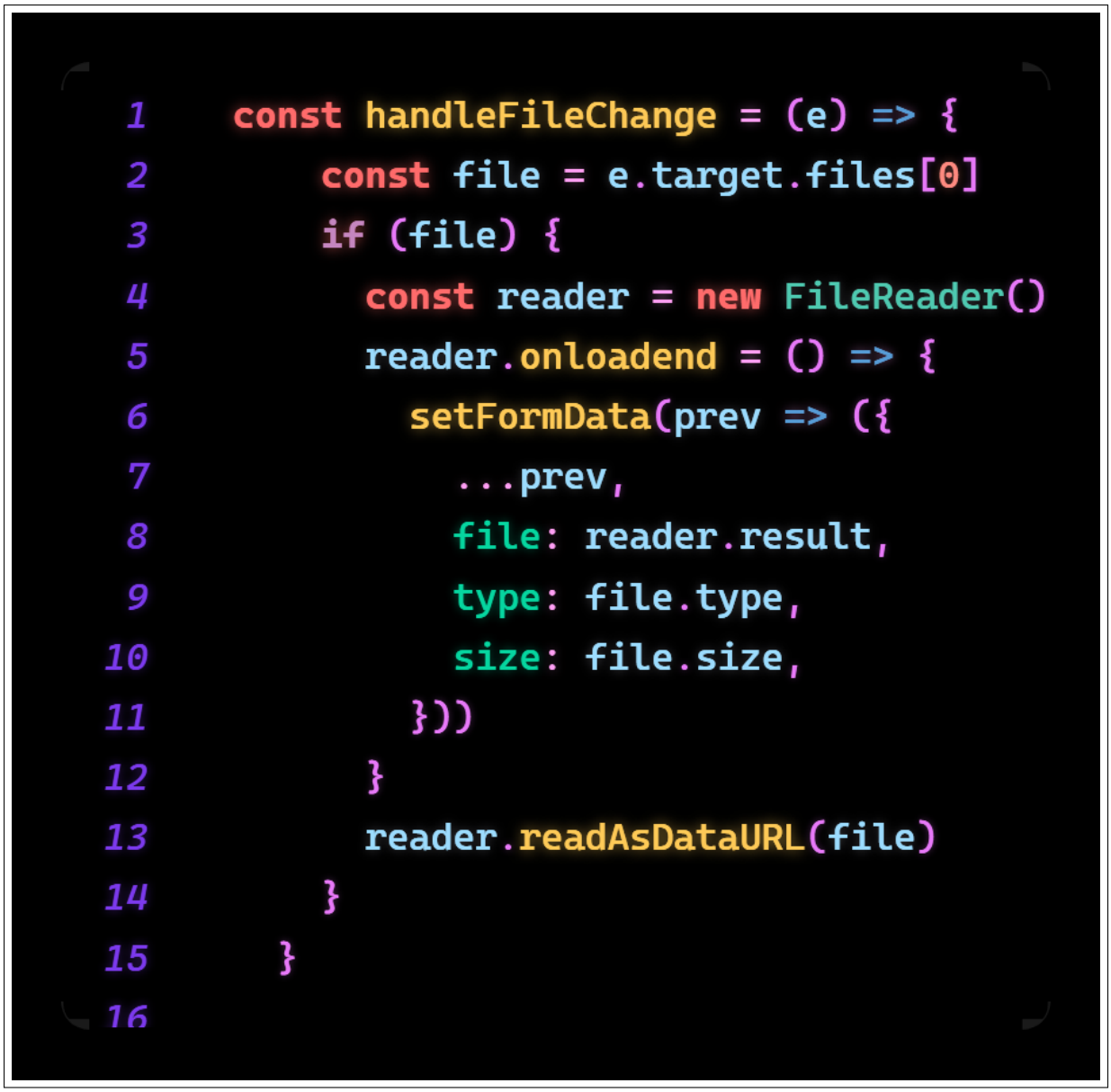
FIGURE 5.26 – Blob traitement BDD

3 Traitement Front-End

```
1    const [formData, setFormData] = useState({
2        title: '',
3        file: null,
4        type: '',
5        size: 0,
6    });
7
```

FIGURE 5.27 – Blob Traitement Front-End Variables d'état

Variables d'état



```
const handleFileChange = (e) => {
    const file = e.target.files[0]
    if (file) {
      const reader = new FileReader()
      reader.onloadend = () => {
        setFormData(prev => ({
          ...prev,
          file: reader.result,
          type: file.type,
          size: file.size,
        }))
      }
      reader.readAsDataURL(file)
    }
  }

```

FIGURE 5.28 – Blob Traitement Front-End Fonction pour le fichier

Fonction pour le fichier

```
const handleSubmit = async (e) => {
    e.preventDefault();

    const res = await fetch("/api/docs/insert", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      }
      body: JSON.stringify({ formData }),
    });

    const data = await res.json();
    alert(data.message);
  };

```

FIGURE 5.29 – Blob Traitement Front-End Fonction pour la soumission

Fonction pour la soumission

```jsx
return (
    <form onSubmit={handleSubmit} className="p-4 space-y-4">
      <input
        type="text"
        placeholder="Titre"
        value={formData.title}
        onChange={(e) => setFormData(prev => ({...prev, title: e.target.value}))}
        className="border p-2 w-full"
      />
      <input
        type="file"
        onChange={handleFileChange}
        className="border p-2 w-full"
      />
      <button type="submit" className="bg-blue-500 text-white px-4 py-2 rounded">
        Envoyer
      </button>
    </form>
  );

```

FIGURE 5.30 – Blob Traitement Front-End Partie JSX

Partie JSX

4 Traitement Back-End

```
1   import db from '../_lib/connect'
2
3   export default async function handler(req, res) {
4     const { title, file, type, size } = req.body;
5
6     await db.execute(`
7       INSERT INTO Docs (title, file, type, size)
8       VALUES (?, ?, ?, ?)
9     `, [title, file, type, size]);
10
11    res.status(201).json({ message: 'Succès !'});
12  }
13
```

FIGURE 5.31 – Blob Traitement Back-End

5 BodyParser et gestion de taille limite

Parfois, l'upload du fichier se block par le middleware de parsing du body à cause de la taille maximale par défaut de **1mb**, pour résoudre ce problème on ajoute dans notre l'api :

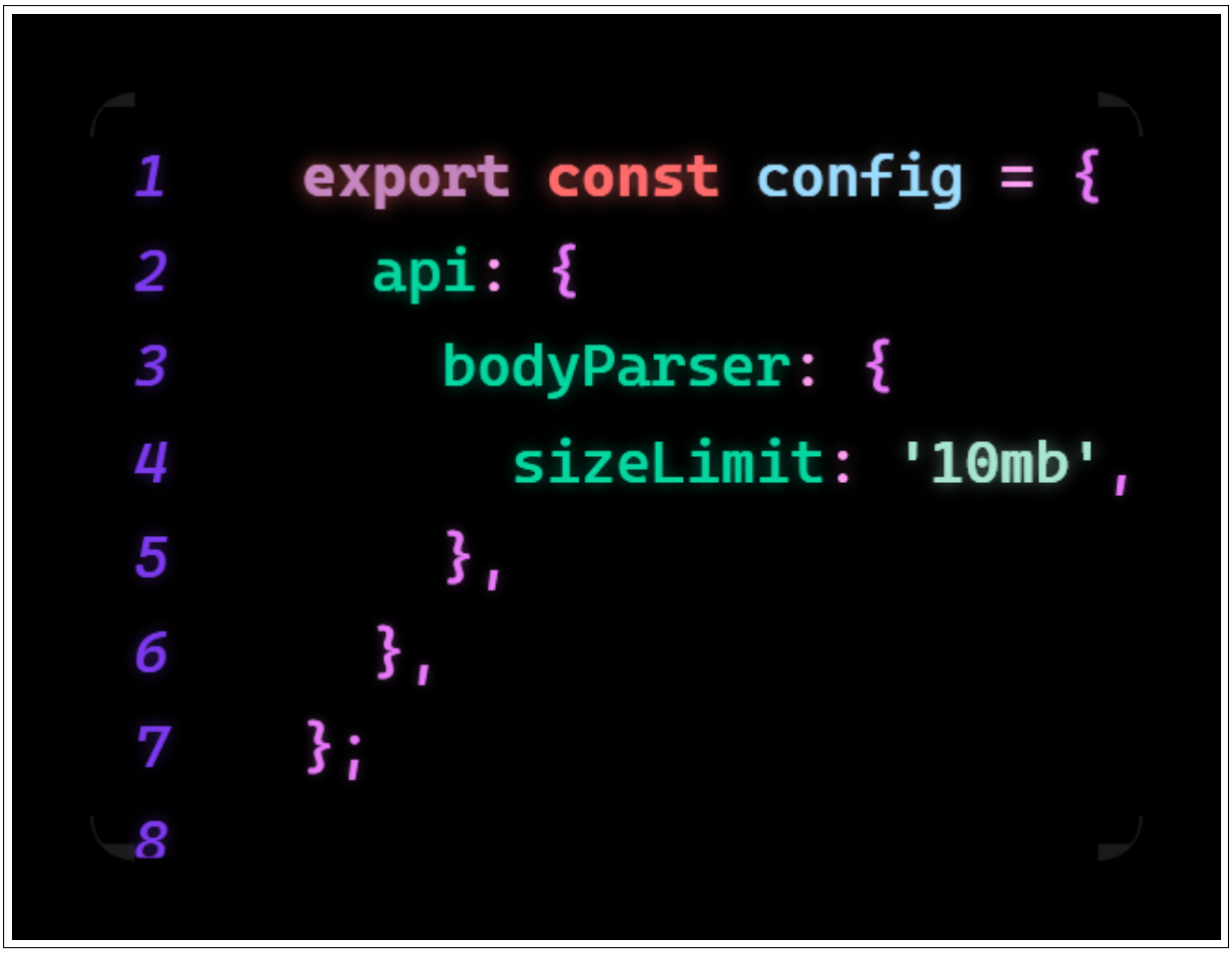


FIGURE 5.32 – BodyParser

6 Contrôle du progès d'upload

Pour faire un progès d'upload avec un pourcentage, on utilise le **XMLHttpRequest** ou bien **Axios** à la place de **fetch** :

```
const handleSubmit = async (e) => {
  e.preventDefault();

  try {
    const res = await axios.post('/api/docs/insert', formData, {
      headers: {
        'Content-Type': 'application/json'
      },
      onUploadProgress: (progressEvent) => {
        const percentCompleted = Math.round(
          (progressEvent.loaded * 100) / progressEvent.total
        );
        setUploadProgress(percentCompleted); // Traitement d'affichage après
      }
    });
  } catch (error) {
    console.error(error);
  }
}
```

FIGURE 5.33 – Contrôle du progès d'upload

7 Récupération des données

7.1 Traitement Back-End

Dans **/api/docs/getAll.js** :

```
export default async function handler(req, res) {
  try {
    const [docs] = await db.execute('SELECT * FROM Docs');

    const formattedDocs = docs.map(doc => ({
      ...doc,
      file: doc.file
        ?
          `data:${doc.type || 'application/octet-stream'}
          ; base64,${Buffer.from(doc.file).toString('base64')}`
        : null,
    }));

    res.status(200).json(formattedDocs);
  } catch (error) {
    res.status(500).json({ message: 'Erreur serveur' });
  }
}

```

FIGURE 5.34 – Récupération des données BLOB

7.2 Traitement Front-End

Après récupération des données avec l'api, on peut afficher le fichier si il est image comme ça :

```
<img src={doc.img || '/default.png'}>

```

FIGURE 5.35 – Affichage d'image BLOB

Parfois ceci ne fonctionne pas car la chaîne est encodée en Base64 et on veut la convertir en données lisibles, pour cela on utilise **atob** sur la partie après " ; base64," dans la chaîne **data :image/png ; base64,iVBORw0KGgoAAAANS...** :

```
<img src={doc.img ? atob(doc.img.split(',')[1]) : '/default.png'}>
```

FIGURE 5.36 – Affichage d'image base64

Pour le cas de n'importe quel type de fichiers :

```
{doc.type.startsWith('image/') ? (
  <img src={doc.file} alt={doc.title} />
) : (
  <a href={doc.file} download={doc.title}>
    Télécharger {doc.title}
  </a>
)}
```

FIGURE 5.37 – Affichage de n'importe quel fichier

8 Téléchargement d'un fichier

8.1 Traitement Back-End

```
export default async function handler(req, res) {
  try {
    const [docs] = await db.execute('SELECT * FROM Docs WHERE id=1');
    const doc = docs[0];

    res.setHeader('Content-Type', doc.type || 'application/octet-stream');
    res.setHeader('Content-Disposition', `attachment; filename=My_Doc.${doc.type.split('/')[1] || 'bin'}`);
    res.send(doc.file); // renvoyer directement le buffer binaire
  } catch (error) {
    res.status(500).json({ message: 'Erreur serveur' });
  }
}
```

FIGURE 5.38 – Téléchargement d'un fichier - Traitement Back-End

8.2 Traitement Front-End

```
1   const handleDownload = async () => {
2     try {
3       const response = await fetch('/api/docs/getOne');
4
5       const blob = await response.blob();
6       const url = window.URL.createObjectURL(blob);
7       const a = document.createElement('a');
8       a.href = url;
9
10      const contentType = response.headers.get('Content-Type');
11      const extension = contentType?.split('/')[1] || 'bin';
12
13      a.download = `My_Doc.${extension}`;
14      document.body.appendChild(a);
15      a.click();
16
17      document.body.removeChild(a);
18      window.URL.revokeObjectURL(url);
19    } catch (error) {
20      console.error('Error downloading the file:', error);
21    }
22  };
23
```

FIGURE 5.39 – Téléchargement d'un fichier - Traitement Front-End

V Authentification

1 Middlewares d'authentification avec JWT

Un Middleware est un morceau de code intermédiaire qui s'exécute entre la requête du client et la réponse du serveur dans une application web. Il sert souvent à traiter, modifier, filtrer ou gérer les requêtes et réponses, par exemple pour gérer l'authentification, la gestion des erreurs, le parsing des données, ou le logging. Dans Nextjs on peut les mettre dans le dossier **middlewares**

```
import jwt from "jsonwebtoken";

const JWT_SECRET = process.env.JWT_SECRET;

export function verifyAuth(req, res) {
  const token = req.cookies?.adminToken;

  if (!token) {
    return null;
  }

  try {
    const admin = jwt.verify(token, JWT_SECRET);
    return {
      id: 1,
    };
  } catch (error) {
    console.error("Invalid token", error);
    return null;
  }
}
```

FIGURE 5.40 – Middlewares d'authentification avec JWT

2 Login et déclanchement d'une session

```
import jwt from "jsonwebtoken";
import { serialize } from "cookie";

export default async function handler(req, res) {
  const { password } = req.body;
  if (password === '123123') {
    const token = jwt.sign({
      nom: 'pahae', age: '22' // Données récupérées d'une requête par exemple
    }, process.env.JWT_SECRET, {
      expiresIn: "7d",
    });

    const serialized = serialize("adminToken", token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === "production",
      sameSite: "strict",
      maxAge: 60 * 60 * 24 * 7,
      path: "/",
    });

    res.setHeader("Set-Cookie", serialized);

    return res.status(200).json({ message: 'Connexion réussie' });
  }
  else
    return res.status(400).json({ message: 'Mot de passe incorrect' });
}
```

FIGURE 5.41 – Login et déclanchement d'une session

3 Logout et destruction de la session

```
import { serialize } from 'cookie';

export default function handler(req, res) {
  if (req.method === 'POST') {
    res.setHeader('Set-Cookie', serialize('adminToken', '', {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      path: '/',
      expires: new Date(0),
    }));
    return res.status(200).json({ message: 'Logout successful' });
  }
  res.setHeader('Allow', ['POST']);
  res.status(405).json({ error: `Method ${req.method} not allowed` });
}

```

FIGURE 5.42 – Logout et destruction de la session

4 Récupération du compte

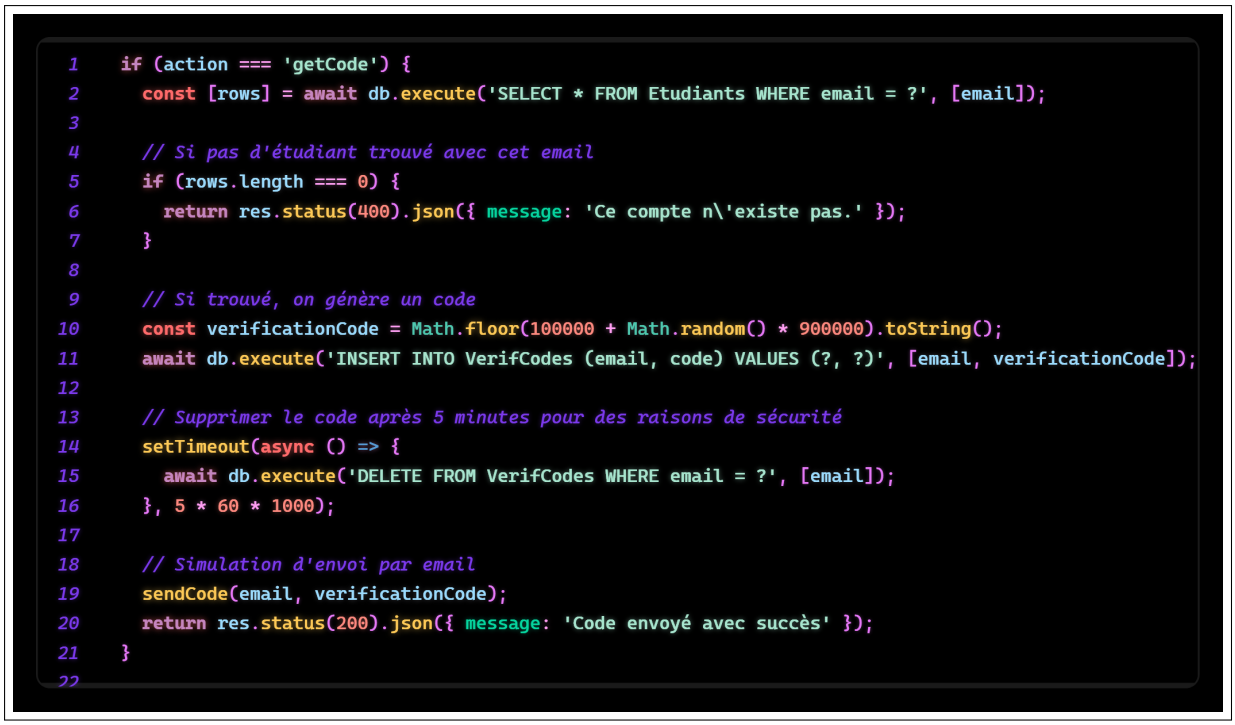
4.1 Table de BDD pour les codes de vérification

```
CREATE TABLE VerifCodes (
  code VARCHAR(255) PRIMARY KEY,
  email VARCHAR(255) NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

FIGURE 5.43 – Table de BDD pour les codes de vérification

4.2 Algorithme à procéder

On demande à l'utilisateur d'entrer son email, si il n'est pas valide on renvoie une erreur, sinon on génère un code et on l'insère dans **VerfiCodes** :



```
if (action === 'getCode') {
  const [rows] = await db.execute('SELECT * FROM Etudiants WHERE email = ?', [email]);

  // Si pas d'étudiant trouvé avec cet email
  if (rows.length === 0) {
    return res.status(400).json({ message: 'Ce compte n\'existe pas.' });
  }

  // Si trouvé, on génère un code
  const verificationCode = Math.floor(100000 + Math.random() * 900000).toString();
  await db.execute('INSERT INTO VerifCodes (email, code) VALUES (?, ?)', [email, verificationCode]);

  // Supprimer le code après 5 minutes pour des raisons de sécurité
  setTimeout(async () => {
    await db.execute('DELETE FROM VerifCodes WHERE email = ?', [email]);
  }, 5 * 60 * 1000);

  // Simulation d'envoi par email
  sendCode(email, verificationCode);
  return res.status(200).json({ message: 'Code envoyé avec succès' });
}
```

FIGURE 5.44 – Génération de code de récupération

On demande à l'utilisateur d'entrer le code reçu, si il n'est pas valide ou expiré on renvoie une erreur, sinon on lui permet de continuer vers l'étape suivante :

```
if (action === 'verifyCode') {
  const [rows] = await db.execute('SELECT * FROM VerifCodes WHERE email = ? AND code = ?', [email, code]);
  if (rows.length === 0) {
    return res.status(400).json({ message: 'Code invalide' });
  }
  return res.status(200).json({ message: 'Code vérifié avec succès' });
}
```

FIGURE 5.45 – Vérification du code de récupération

On demande à l'utilisateur d'entrer un nouveau mot de passe, :

```
if (action === 'resetPassword') {
  const hashedPassword = await bcrypt.hash(password, 10);
  await db.execute('UPDATE Etudiants SET password = ? WHERE email = ?', [hashedPassword, email]);
  return res.status(200).json({ message: 'Mot de passe réinitialisé avec succès' });
}
```

FIGURE 5.46 – Récupération du mot de passe

5 Validation de Captcha

5.1 Intêret du Captcha

Le Captcha sert à vérifier qu'un utilisateur est bien un humain et non un robot.
Intérêts principaux :

— Empêcher les bots d'envoyer des requêtes automatiques (inscriptions massives, spams, attaques, etc.).

— Protéger les formulaires (connexion, inscription, commentaires) contre les abus.

— Renforcer la sécurité globale des sites web.

C'est une barrière simple mais efficace contre les actions automatisées malveillantes.

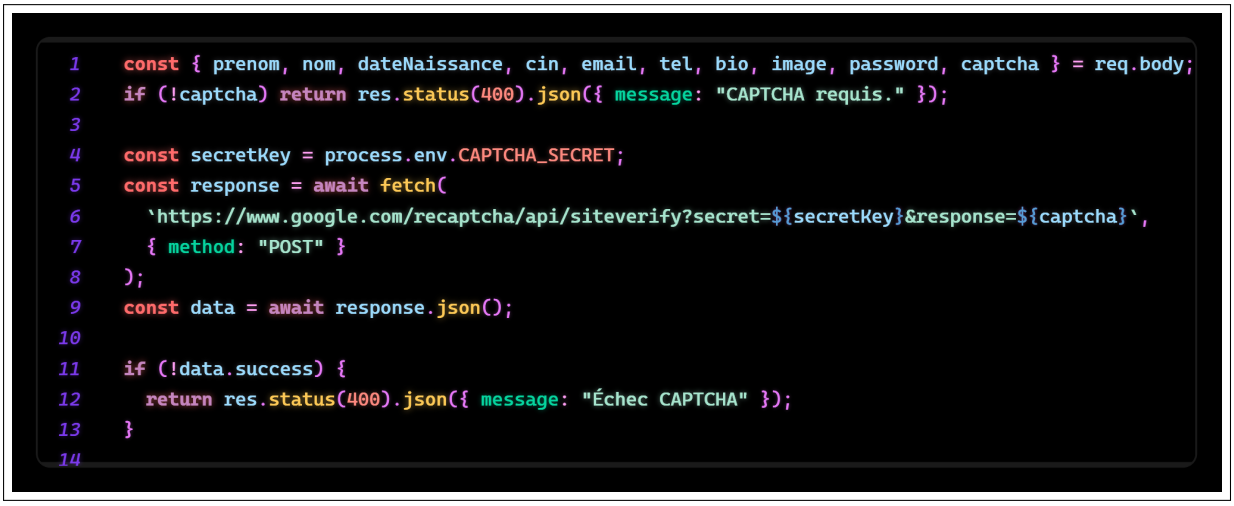
5.2 Implémentation Front-End

```jsx
import ReCAPTCHA from "react-google-recaptcha";

export default function SignIn() {
  const [formData, setFormData] = useState({});
  const [captcha, setCaptcha] = useState(null);
  const handleCaptcha = (value) => {
    setCaptcha(value);
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!captcha) { alert('erreur'); return; }

    const response = await fetch('/api/_auth/userRegister', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        ...formData,
         captcha: captcha
      }),
    });
    // Autres traitements
  };

  return (
    <>
      // Eléments du formulaire

      <ReCAPTCHA
        sitekey="6LdR5lQrAAAAAPonFpFT2ahDbEo97ia30GBPqOGP"
        onChange={() }
      />
    </>
  );
}
```

FIGURE 5.47 – Captcha Front-End

5.3 Implémentation Back-End

D'abord il faut aller à le site officiel et récupérer un clé API, puis dans votre api de SignIn :



```
const { prenom, nom, dateNaissance, cin, email, tel, bio, image, password, captcha } = req.body;
if (!captcha) return res.status(400).json({ message: "CAPTCHA requis." });

const secretKey = process.env.CAPTCHA_SECRET;
const response = await fetch(
  `https://www.google.com/recaptcha/api/siteverify?secret=${secretKey}&response=${captcha}`,
  { method: "POST" }
);
const data = await response.json();

if (!data.success) {
  return res.status(400).json({ message: "Échec CAPTCHA" });
}
```

FIGURE 5.48 – Captcha Back-End

6 Récupération et vérification de la session dans les pages

Pour exploiter les sessions en Front-End, il faut utiliser la fonction **verifyAuth** qu'on a créé dans le dossier **middelwares**, mais la session est stockée côté serveur, c'est pour cela on doit la récupérer depuis là avec **getServerSideProps**. On peut après faire passer la session en page-props suivant cette logique :

```
import { verifyAuth } from '@/middlewares/Etudiant'

export default function Profile({ session }) {
  // Exemple de traitement en utilisant la session :
  useEffect(() => {
    if(!session) window.location.href = "/login"
    else alert(`Bonjour, Monsieur ${session.nom} !`)
  }, []);
}

export async function getServerSideProps({ req, res }) {
  const user = verifyAuth(req, res)

  if (user) {
    return {
      props: {
        session: {
          id: user.id,
          nom: user.nom,
          // Autres données selon la logique d'authentification en Back-End
        }
      },
    }
  }

  return { props: { session: null } }
}
```

FIGURE 5.49 – Vérification de la session dans les pages

VI Fournisseurs d'accès

1 Login avec Google

1.1 Implémentation Front-End

Pour implémenter un système de login avec Google, il faut suivre un ensemble des étapes :

Obtenir les clés API : accéder à Google Cloud Console pour avoir les clés API nécessaires puis les mettre dans le fichier **.env** comme suit :

```
NEXT_PUBLIC_GOOGLE_CLIENT_ID=...
```

Inclure le script google dans .jsx dans le fichier principal **_app.jsx**, il faut mettre :

```jsx
import Script from "next/script";

export default function MyApp({ Component, pageProps }) {
  return (
    <>
      <Component {...pageProps} />
      <Script
        src="https://accounts.google.com/gsi/client"
        strategy="beforeInteractive"
        onLoad={() => console.log("Google Sign-In script loaded")}
      />
    </>
  );
}

```

FIGURE 5.50 – Login avec Google Script

```jsx
import GoogleLoginBtn from "../components/GoogleLoginBtn";

export default function Login() {
  const handleCredentialResponse = async (response) => {
    try {
      const { data } = await axios.post("/api/auth/googleLogin", {
        token: response.credential,
      });
      window.location.href = "/"
    } catch (err) {
      setError("Google authentication failed");
    }
  };

  return <GoogleLoginBtn onLoginSuccess={handleCredentialResponse} className="hidden" />
}

```

FIGURE 5.51 – Login avec Google Front-End

Dans le fichier Login.jsx :

```
import { useEffect } from "react";

export default function GoogleLoginBtn({ onLoginSuccess }) {
  useEffect(() => {
    if (window.google) {
      window.google.accounts.id.initialize({
        client_id: process.env.NEXT_PUBLIC_GOOGLE_CLIENT_ID,
        callback: onLoginSuccess,
      });
      window.google.accounts.id.renderButton(
        document.getElementById("googleSignInDiv"),
        { theme: "outline", size: "large" }
      );
    }
  }, [onLoginSuccess]);

  return <div id="googleSignInDiv"></div>;
}

```

FIGURE 5.52 – Login avec Google Bouton

Voici le component GoogleLoginBtn :

1.2 Implémentation Backend-End

```
import { OAuth2Client } from "google-auth-library";
const client = new OAuth2Client(process.env.NEXT_PUBLIC_GOOGLE_CLIENT_ID);

export default async function handler(req, res) {
  const { token } = req.body;

  try {
    const ticket = await client.verifyIdToken({
      idToken: token,
      audience: process.env.NEXT_PUBLIC_GOOGLE_CLIENT_ID,
    });

    const payload = ticket.getPayload();
    const { email, name, picture } = payload;
    // Autres traitements avec l'email, le name la picture ...

    return res.status(200).json({ message: "Login successful" });
  } catch (err) {
    console.error(err);
    return res.status(401).json({ message: "Invalid Google token" });
  }
}
```

FIGURE 5.53 – Login avec Google Back-End

2 Login avec Facebook

2.1 Implémentation Front-End

Après avoir obtenu la clé API depuis Facebook Developers, voici les étapes à suivre :

```jsx
import FacebookLoginButton from "../components/FacebookLoginButton";
import axios from "axios";

export default function Login() {
  const handleFacebookLogin = async (accessToken) => {
    try {
      const { data } = await axios.post("/api/auth/facebookLogin", { accessToken });
      window.location.href = "/";
    } catch (err) {
      console.error(err);
    }
  };

  return (
    <div className="min-h-screen flex justify-center items-center">
      <FacebookLoginButton onSuccess={handleFacebookLogin} />
    </div>
  );
}
```

FIGURE 5.54 – Login avec Facebook Front-End

Dans le fichier Login.jsx

2.2 Implémentation Backend-End

```jsx
import axios from "axios";

export default async function handler(req, res) {
  const { accessToken } = req.body;

  try {
    const fbResponse = await axios.get(`https://graph.facebook.com/me?fields=id,name,email,picture&access_token=${accessToken}`);
    const { id, name, email, picture } = fbResponse.data;

    // Autres traitements avec l'id, le name...

    return res.status(200).json({ message: "Login successful" });
  } catch (error) {
    console.error(error);
    return res.status(401).json({ message: "Invalid Facebook token" });
  }
}
```

FIGURE 5.55 – Login avec Facebook Back-End

3 Login avec Github

3.1 Implémentation Front-End

Après avoir obtenu la clé API depuis Github Developers, voici les étapes à suivre :

```jsx
export default function GitHubLoginButton() {
  const handleLogin = () => {
    const clientId = process.env.NEXT_PUBLIC_GITHUB_CLIENT_ID;
    const redirectUri = "http://localhost:3000/api/auth/github/callback";
    window.location.href = `https://github.com/login/oauth/authorize?client_id=${clientId}&redirect_uri=${redirectUri}&scope=user:email`;
  };

  return (
    <button onClick={handleLogin} className="bg-gray-800 text-white px-4 py-2 rounded">
      Login with GitHub
    </button>
  );
}

```

FIGURE 5.56 – Login avec Github Front-End

Dans le fichier Login.jsx

3.2 Implémentation Backend-End

screens/Login avec Github Back-End.png

FIGURE 5.57 – Login avec Github Back-End

Chapitre 6 : Communications en temps réel avec Socket.IO

Chapitre 6

Communications en temps réel avec Socket.IO

I Principe et logique des sockets

1 Problème de communication via HTTP

HTTP est un protocole de communication uniderctionnel, c'est à dire il ne peut pas être utiliser pour un besoin à temps réel. Mais une solution a été proposée : Le **HTTP Pooling**

1.1 HTTP Pooling

C'est l'envoi des requêtes HTTP régulièrement suivant cet algorithme là (pour un exemple de système de messagerie) :

— **Etape 1 :** Appeler la fonction fetchMsgs() qui récupère les messages avec une API (requête HTTP).

- **Etape 1.1 :** Si la récupération échoue On signale une erreur à l'utilisateur → Fin de l'algorithme.
- **Etape 1.2 :** Si la récupération réussie → On affiche les messages .
  - **Etape 1.2.1 :** On rappelle la fonction fetchMsgs().

C'est comme une fonction récursive ! mais asynchrone. Voici une application de cet algorithme :

```jsx
import { useState, useEffect } from "react";

export default function Login() {
    const [msgs, setMsgs] = useState([]); // D'abord on n'a pas de messages
    useEffect(() => fetchMsgs(), []) // Appeler fetchMsgs une seul fois

    const fetchMsgs = async () => {
        const res = await fetch('/api/getMsgs')
        if (res.ok) {
            const data = res.json();
            setMsgs(data.msgs || []);
            fetchMsgs(); // HTTP Pooling
        }
        else console.log('Erreur -> Fin du programme')
    }

    return (
        <> { msgs.map(msg => (<p>{msg.content}</p>)) } </>
    );
}

```

FIGURE 6.1 – HTTP Pooling

1.2 Inconvéniant de HTTP Pooling

Le client envoie des requêtes à intervalles réguliers, même s'il n'y a aucune donnée disponible, cela sature inutilement le serveur et utilise plus de bande passante. En plus, si la fréquence d'interrogation est trop basse, l'information peut arriver tard.

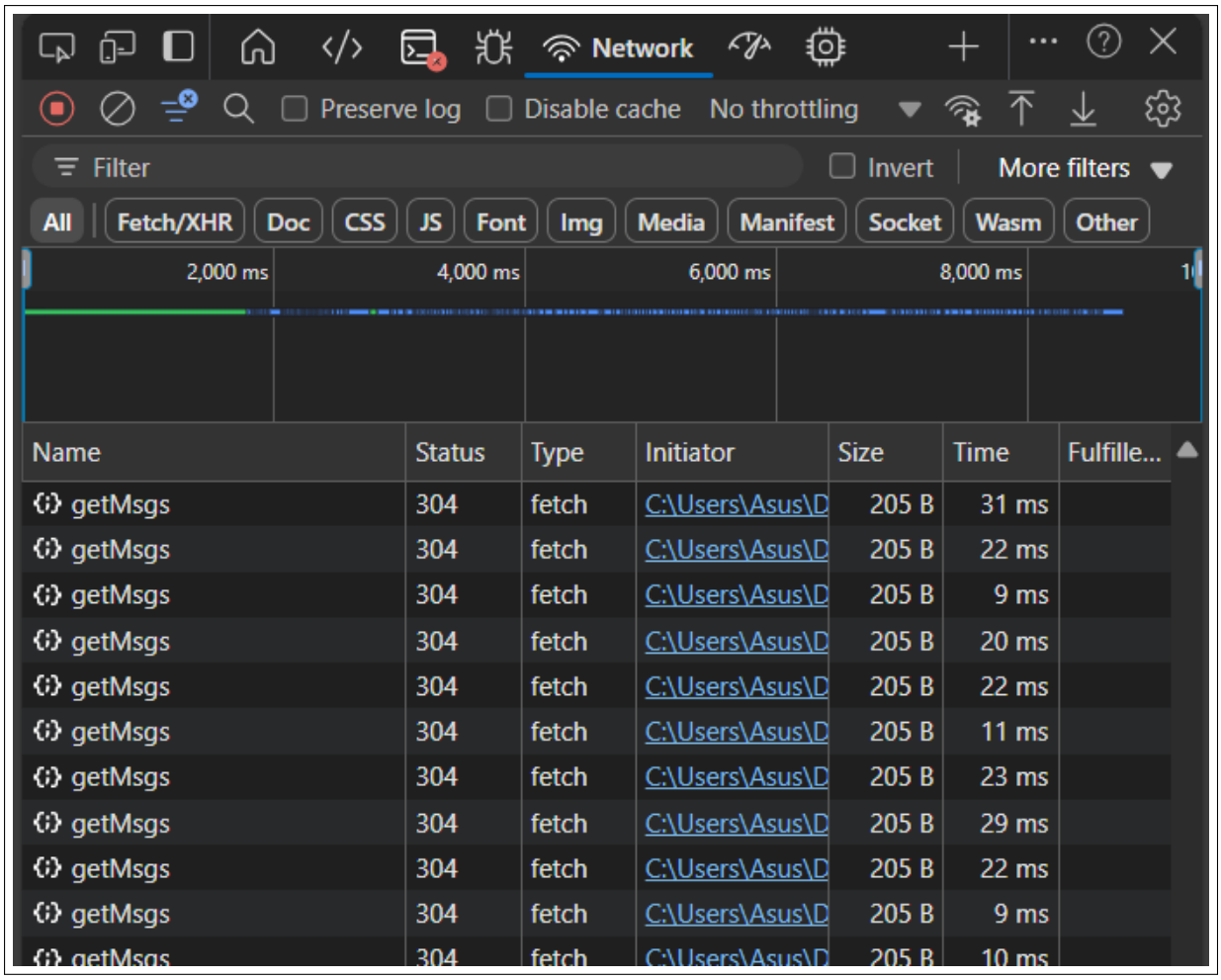


FIGURE 6.2 – Inconvéniant de HTTP Pooling

2 Solution : TCP/IP

Contrairement à HTTP, TCP maintient une connexion ouverte qui autorise l'envoi et la réception de données en continu, sans avoir besoin de réinitialiser la communication à chaque échange. La communication basée sur TCP/IP permet donc :

— Un échange de données en temps réel (faible latence).

— Une consommation de bande passante optimisée, car les connexions ne sont pas réinitialisées.

— Une interaction **bidirectionnelle** continue entre client et serveur.

2.1 Principe de fonctionnement de de Socket.IO

L'idée de Socket.IO c'est d'échanger entre les clients 4 parties pricipales via TCP/IP :

— **Connexion :** Socket.IO établit une connexion WebSocket, avec fallback (cas de connexion échouée) → HTTP Polling si nécessaire.

— **Evénements :** Échanges de messages, notification, etc. via un système d'événements (Quand on reçoit un évévenement d'un client on le diffuse à tous les autres).

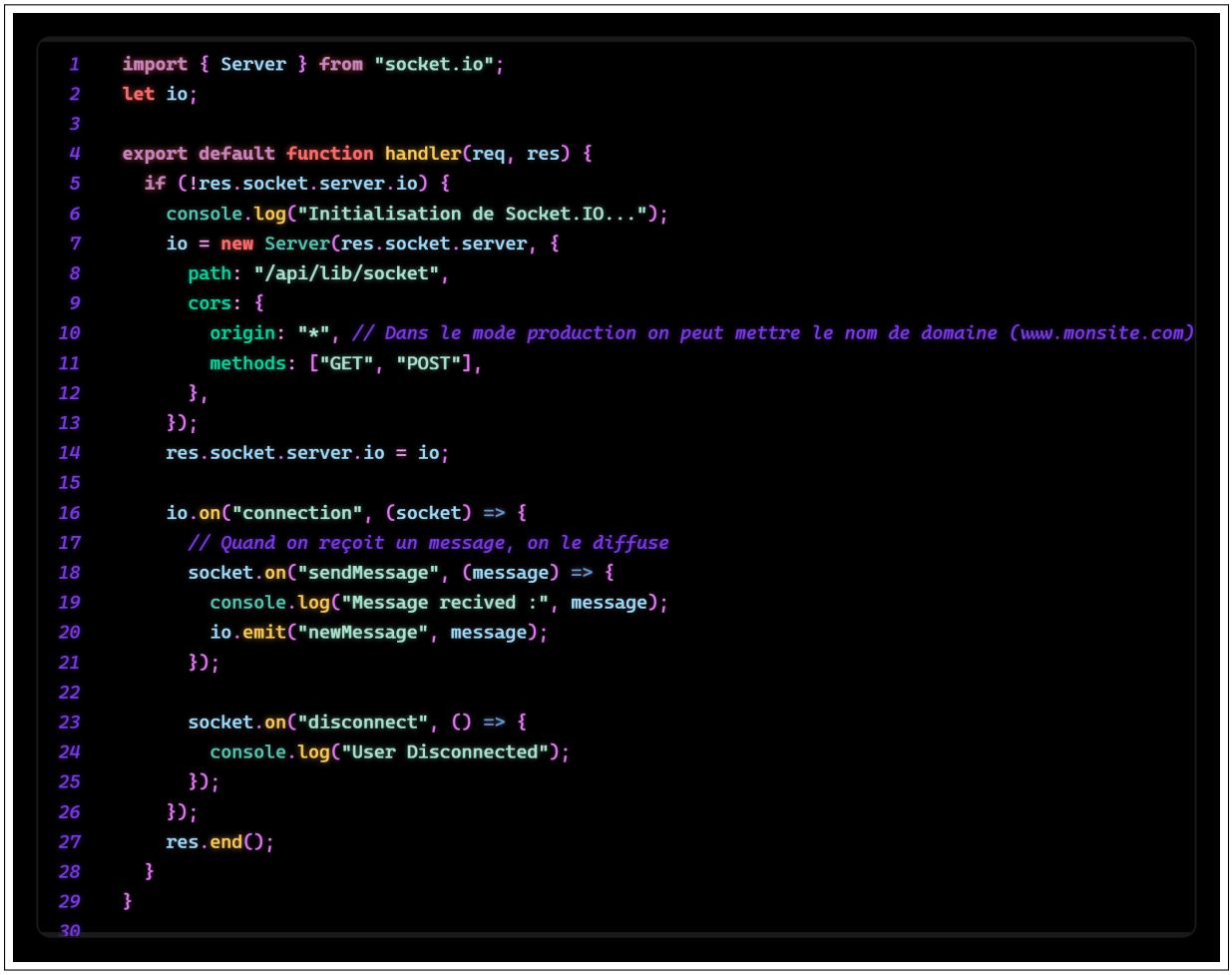
— **Déconnexion :** Quand un client se déconnecte.

— **Reconnexion :** Tentative automatique si la connexion est perdue.

II Implémentation d'un système de messagerie

1 Configuration d'un serveur WebSocket

Le serveur est configuré pour accepter les connexions des clients et gérer les événements de communication.
Dans /api/_lib/socket :



```
import { Server } from "socket.io";
let io;

export default function handler(req, res) {
  if (!res.socket.server.io) {
    console.log("Initialisation de Socket.IO...");
    io = new Server(res.socket.server, {
      path: "/api/lib/socket",
      cors: {
        origin: "*", // Dans le mode production on peut mettre le nom de domaine (www.monsite.com)
        methods: ["GET", "POST"],
      },
    });
    res.socket.server.io = io;

    io.on("connection", (socket) => {
      // Quand on reçoit un message, on le diffuse
      socket.on("sendMessage", (message) => {
        console.log("Message recived :", message);
        io.emit("newMessage", message);
      });

      socket.on("disconnect", () => {
        console.log("User Disconnected");
      });
    });
    res.end();
  }
}
```

FIGURE 6.3 – Configuration d'un serveur WebSocket

2 Initialisation des sockets

Côté client, nous initialisons la connexion avec le serveur Socket.IO. C'est à ce moment que nous définissons les événements que le client doit écouter, tels que la réception de nouveaux messages.

```
useEffect(() => initSocket(), []);

async function initSocket() {
  await fetch("/api/_lib/socket");
  socket = io({
    path: "/api/_lib/socket",
  });
  socket.on("connect", () => {
    console.log("Socket connected:", socket.id);
  });
  socket.on("newMessage", (message) => {
    fetchMessages();
  });
  socket.on("disconnect", () => {
    console.log("Socket disconnected");
  });
}
```

FIGURE 6.4 – Initialisation des sockets

Dans le fichier Messages.jsx :

3 Récupération des messages

Cette partie assure la récupération des messages existants avec l'API.

```
useEffect(() => fetchMessages(), []);

const fetchMessages = async () => {
  const res = await fetch('/api/msgs/getAll');
  const data = res.json();
  setMessages(data || []); // Ou autres traitements...
};

```

FIGURE 6.5 – Récupération des messages

4 Emission d'un message

Une fois notre message est envoyé avec succès (**response.ok**), on déclanche l'événement **sendMessage**, et puis l'événement newMessage **newMessage** va être émise automatiquement par le serveur pour notifier les autres utilisateurs (voir la configuration du serveur).

```
const sendMessage = async (msg) => {
  const response = await fetch("/api/msg/sendMsg", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ msg }),
  });

  if (response.ok) {
    // On déclanche l'événement sendMessage
    socket.emit("sendMessage", msg);
  } else {
    console.error("Error sending message");
  }
};

```

FIGURE 6.6 – Emission d'un message

5 Réception d'un message

Une fois le **newMessage** est diffusé, il va être capté par (Voir l'initialisation des sockets) :

```
socket.on("newMessage", (message) => {
  fetchMessages();
});
```

III Implémentation d'un système de notifications

1 Emission d'une notification

Dans la configuration du serveur Websockets, on ajoute une emission de notification :

```
io.on("connection", (socket) => {
  socket.on("sendMessage", (message) => {
    console.log("Message recived :", message);
    io.emit("newMessage", message);
  });

  socket.on("newNotification", ({ to, msg }) => {
    io.emit("sendNotification", {
      to: to,
      content: `Vous avez un nouveau message: ${msg}`,
    });
  });

  socket.on("disconnect", () => {
    console.log("Disconnected");
  });
});
```

FIGURE 6.7 – Emission d'une notification

Puis dans la fonction d'envoi du message, on émet une notification :

```
1   if(response.ok) {
2     socket.emit("newMessage", msg);
3     socket.emit("newNotification", { to: userID, msg: msg });
4   }
5
```

FIGURE 6.8 – Emission d'une notification après envoi

2 Réception d'une notification

2.1 Notification interne (dans le navigateur)

On passe maintenant au fichier principal **_app.jsx**.

Pourquoi ? → Pourque la notification apparaîsse dans n'importe quelle page dans notre application, sinon il va apparaîte seulement si on est sur la page des messages.

Quoi faire ? → On écoute à l'événement **newNotification**, puit on affiche une notification avec **ReactToast**.

```jsx
// Partie Importations
import { toast, ToastContainer } from "react-toastify";
import "react-toastify/dist/ReactToastify.css";
import { io } from "socket.io-client";
let socket;

// Partie JS
useEffect(() => {
  initSocket();
  if(socket) {
    socket.on("newNotification", (notif) => {
      if(notif.to === session.id) {
        const notificationSound = new Audio("/sounds/ma_notif.wav");
        notificationSound.play().catch((e) => console.error("Erreur lecture audio:", e));
        toast.info(data.message, {
          position: "top-right",  // Ajouter la position dans la page
          autoClose: 5000, // Fermer automatiquement après 5 secondes
          hideProgressBar: false,  // Afficher la bar de progrès
          closeOnClick: true,  // Se ferme si on clique
          pauseOnHover: true,  // La bar de progrès s'arrête si on hover
          draggable: true,  // Peut être déplacée avec la souris
        });
      }
    });
  }
  return () => {
    if (socket) {
      socket.off("notification");
    }
  };
}, [session]);

// Partie JSX
return (
  <>
    <ToastContainer />
    <Component {...pageProps} />
  </>
);
```

FIGURE 6.9 – Réception d'une notification interne

2.2 Notification externe (dans l'apparail)

une notification externe s'affiche en dehors du navigateur, son style n'est pas configurable car elle dépend du système d'exploitation (Windows, Android...).

```
useEffect(() => {
  initSocket();
  if(socket) {
    socket.on("newNotification", (notif) => {
      if(notif.to === session.id)
        alertNotification("Nouvelle notification", notif.msg, "/logo.png");
    });
  }
}, [session]);

function alertNotification(titre, contenu, imageUrl) {
  if (!("Notification" in window)) { console.error("Ce navigateur ne prend pas en charge les notifications bureau"); return; }

  if (Notification.permission === "default")
    Notification.requestPermission().then(permission => {
      if (permission === "granted") creerNotification();
    });
  else if (Notification.permission === "granted") creerNotification();
  else console.warn("L'utilisateur a refusé les notifications");

  function creerNotification() {
    const options = {
      body: contenu,
      icon: imageUrl || "",
      badge: imageUrl || "",
      vibrate: [200, 100, 200],
      tag: "notification-" + Date.now(),
      renotify: true,
      requireInteraction: true
    };
    const notification = new Notification(titre, options);
    notification.onclick = function () {
      window.focus();
      notification.close();
      console.log("Notification cliquée");
    };
    setTimeout(() => notification.close(), 10000);
  }
}
```

FIGURE 6.10 – Réception d'une notification externe

IV Gestion de salles

Dans Socket.IO, la gestion des salles (rooms) permet de regrouper des clients afin de leur envoyer des messages de manière ciblée. Une salle est une sorte de groupe temporaire auquel les clients peuvent s'abonner ou se désabonner dynamiquement.

1 Concept des salles

Lorsqu'un client se connecte à un serveur Socket.IO, il est par défaut dans une salle unique qui correspond à son identifiant de socket. Cependant, il peut rejoindre une ou plusieurs salles supplémentaires grâce à la méthode `socket.join(room)`. Une salle permet donc de diffuser des messages uniquement aux clients qui y sont inscrits.

2 Diffusion des messages

Il existe plusieurs méthodes pour envoyer des messages dans Socket.IO :

— **Diffusion privée** : envoyer un message uniquement à un client spécifique.

— **Diffusion globale** : envoyer un message à tous les clients connectés.

— **Diffusion ciblée** : envoyer un message uniquement aux clients d'une salle spécifique.

— **Diffusion ciblée (sauf émetteur)** : envoyer un message à une salle spécifique, mais en excluant l'émetteur.

— **Diffusion globale (sauf émetteur)** : envoyer un message à tous les clients connectés, sauf l'émetteur.

Le tableau suivant récapitule les différentes syntaxes et les cibles associées :

Syntaxe	**Cible des messages**
`socket.emit(event, data)`	Seulement le client actuel (émetteur)
`io.emit(event, data)`	Tous les clients connectés au serveur
`socket.join(room)`	Le client rejoint une room
`io.to(room).emit(event, data)`	Tous les clients dans la room
`socket.to(room).emit(event, data)`	Tous les clients dans la room sauf l'émetteur
`socket.broadcast.emit(event, data)`	Tous les clients sauf l'émetteur

TABLEAU 6.1 : Résumé des principales méthodes Socket.IO pour la gestion des salles

Chapitre 7 : Communications en temps réel avec WebRTC

Chapitre 7

Communications en temps réel avec WebRTC

I Pourquoi Next.js ne supporte pas WebRTC ?

Next.js est un framework React qui facilite le rendu côté serveur (Server-Side Rendering - SSR) ainsi que la génération de sites statiques. Cependant, WebRTC est une technologie de communication en temps réel peer-to-peer qui s'exécute exclusivement dans l'environnement client (navigateur).

1 Contraintes techniques

La principale raison pour laquelle Next.js ne supporte pas directement WebRTC est liée à la distinction entre le serveur et le client :

- **Exécution côté serveur (Node.js)** : Le code Next.js s'exécute d'abord sur le serveur, où les API WebRTC ne sont pas disponibles car elles font partie des API du navigateur.
- **Exécution côté client (navigateur)** : WebRTC nécessite l'accès aux objets et méthodes propres au navigateur, tels que `RTCPeerConnection`, `getUserMedia`, etc., qui ne peuvent être utilisés que dans un contexte client.

2 Conséquences pratiques

Cela signifie que toute fonctionnalité utilisant WebRTC doit impérativement être exécutée côté client, par exemple dans des composants React qui sont montés uniquement dans le navigateur. Il est donc nécessaire d'utiliser des mécanismes spécifiques pour différer l'exécution de ce code jusqu'au rendu client, comme le hook `useEffect`.

3 Solution courante

Externaliser la logique WebRTC dans des composants purement client-side.

II Configuration de WebRTC avec Express.js

1 Partie d'initiation Express

D'abord on créé un serveur HTTP écoutant au port 5000 (par exemple), on configure le cors et serveur Socket.IO puis on initialise les rooms à un Map vide :

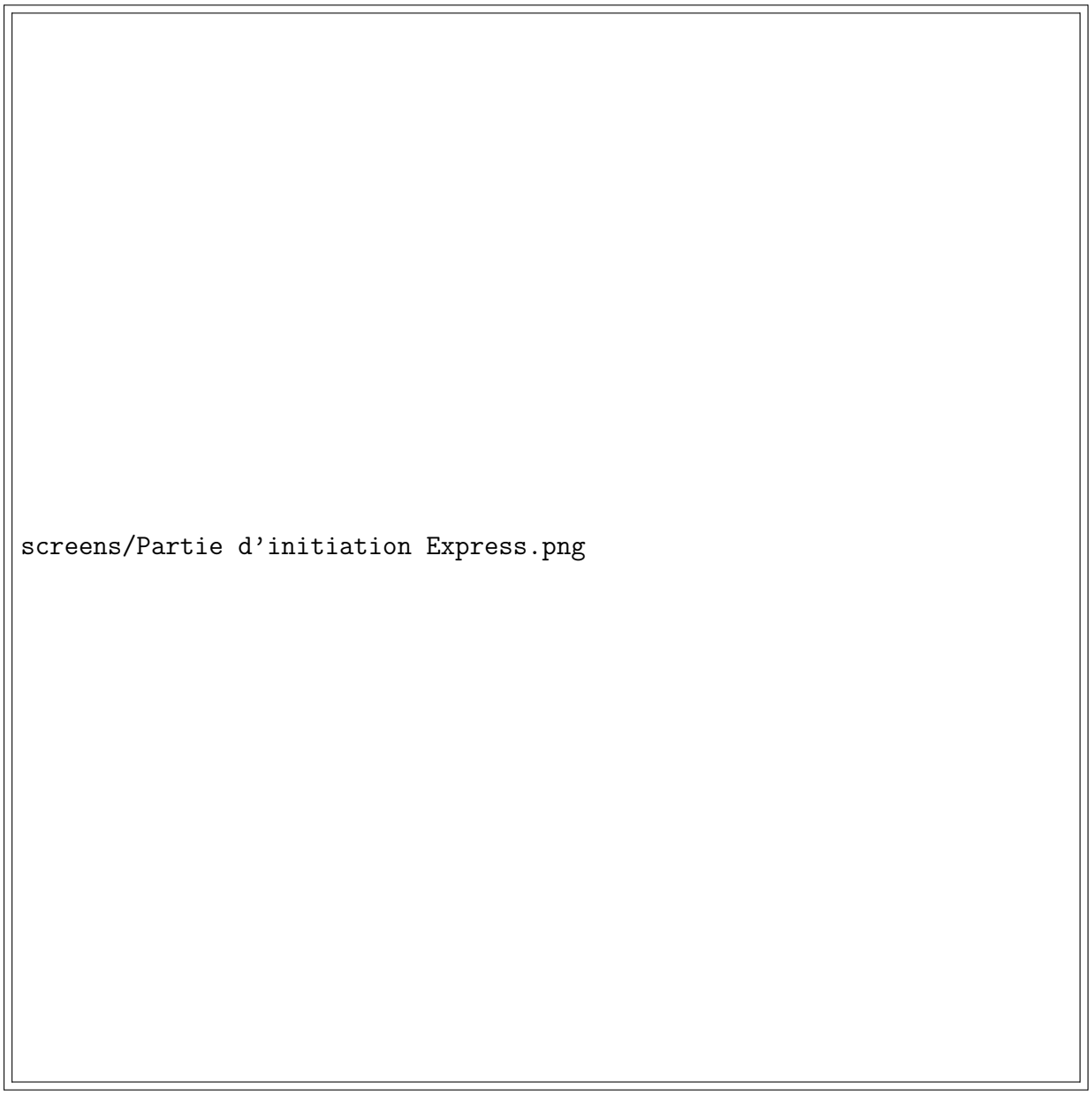


FIGURE 7.1 – Partie d'initiation Express

2 Partie de configuration WebRTC

La configuration d'un serveur WebRTC repose sur 6 phases principales :

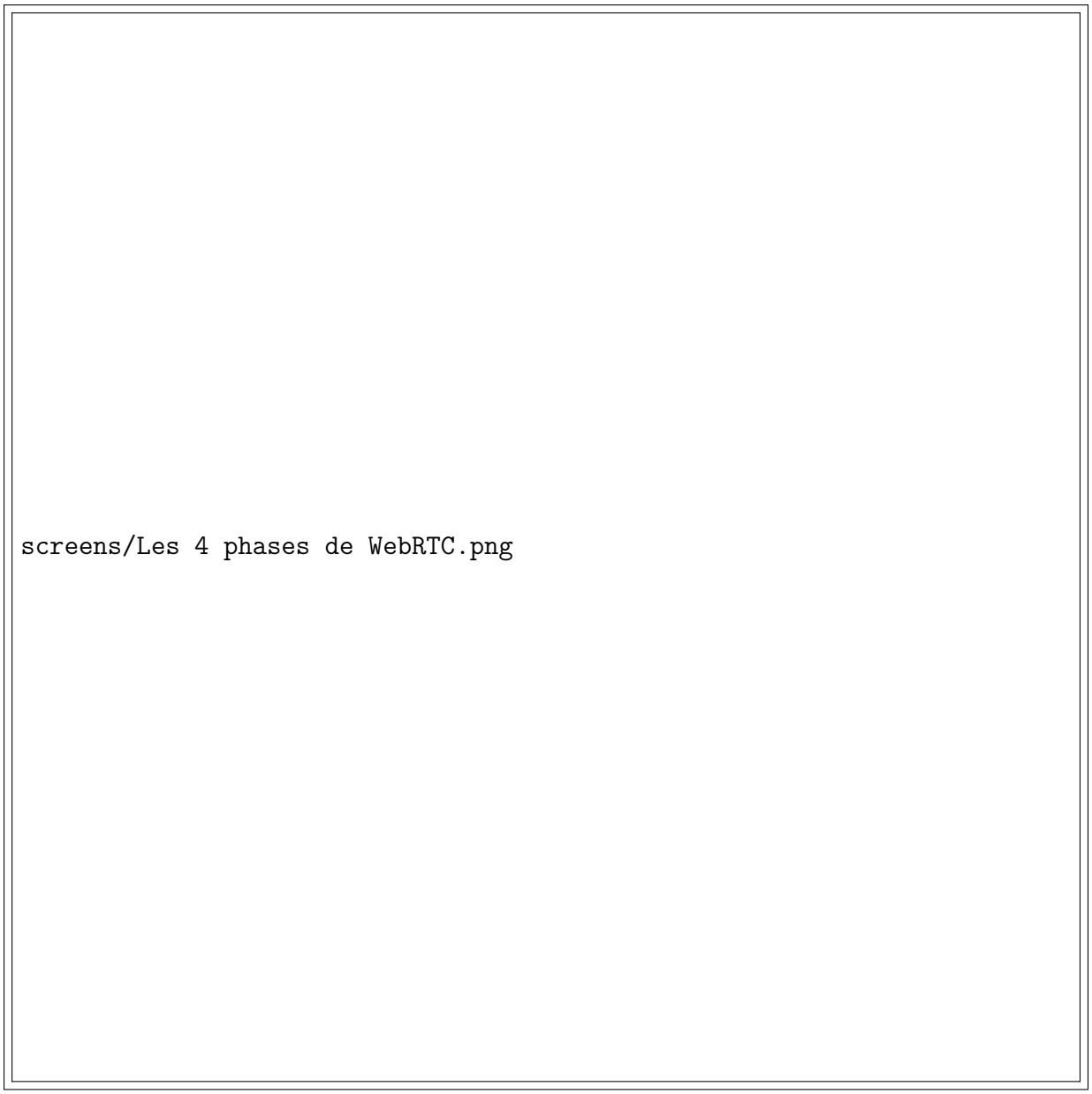


FIGURE 7.2 – Les 4 phases de WebRTC

Voici le but de chacune de ces phases :

Événement	**But principal**
`join-room`	Rejoindre une salle et informer les autres
`webrtc-offer`	Envoyer une offre WebRTC à un autre client
`webrtc-answer`	Répondre à une offre WebRTC
`webrtc-ice-candidate`	Échanger les candidats ICE pour établir la connexion
`* autres événements ... *`	Selon le besoin du projet
`disconnect`	Nettoyer les listes et informer les autres du départ

III Création d'une salle et connexion

Chapitre 8 : APIs et Plugins largrement utilisées

Chapitre 8

APIs et Plugins largrement utilisées

I APIs utiles dans le dévelopement web

1 Gemini API

2 Google-Find-Me API

3 Google-Weather API

4 Firebase API

5 YouTube Data API

6 Unsplash Data API

II Plugins et bibliothèques utiles dans le développement web

1 Facebook-Page Plugin

2 React Toastify

3 React Lucide Icons

4 React Spinners

5 AOS (Animate On Scroll)

6 React Recharts

7 React QRCode

Chapitre 9 : Hébérgement

Chapitre 9

Hébérgement

I Compilation en Next.js

1 Scripts de package.json et commandes npm

Dans un projet Next.js, le fichier `package.json` contient plusieurs scripts utiles pour gérer le cycle de vie de l'application. Voici les scripts les plus courants :

— **`dev`** : Lance l'application en mode développement. La commande associée est :

```
npm run dev
```

Ce mode permet le rechargement à chaud (Hot Reloading) et une compilation lors des modifications.

— **`build`** : Compile le projet pour la production. Cela inclut l'optimisation des pages, le pre-rendering, le minification, etc.

```
npm run build
```

— **`start`** : Démarre le serveur Next.js en mode production après avoir effectué un build.

```
npm start
```

— **`lint`** : Vérifie le code avec ESLint pour détecter les erreurs ou les mauvaises pratiques.

```
npm run lint
```

2 Building du projet

La phase de **build** (ou compilation) prépare l'application Next.js pour un déploiement en production. Cette étape transforme le code source en une version optimisée, prête à être servie par un serveur.

Voici les étapes générales lors du build :

1. Analyse et validation des fichiers avec ESLint (si activé).
2. Compilation du code TypeScript ou JavaScript.
3. Pré-rendu des pages statiques (SSG) et génération des pages côté serveur (SSR).
4. Optimisation des images, du CSS et du JavaScript.
5. Génération du dossier `.next`, contenant tous les fichiers nécessaires à l'exécution de l'application.

La commande pour lancer cette phase est :

```
npm run build
```

Une fois le build terminé, le projet est prêt à être démarré en production avec :

```
npm start
```

II GitHub

1 Notions de base

— **Dépôt local** : c'est l'environnement Git présent sur votre machine. Il contient tout l'historique des versions du projet et permet de suivre les modifications localement.

— **Dépôt distant** : c'est un dépôt hébergé en ligne (ex. : sur GitHub). Il permet de collaborer avec d'autres développeurs et de stocker le code dans le cloud.

— **Commit** : un commit est une sauvegarde d'un ensemble de modifications. Chaque commit est identifié de manière unique et contient un message descriptif. Il fait partie de l'historique du projet.

— **Branch)** : une branche est une version parallèle du projet. Elle permet de travailler sur une nouvelle fonctionnalité ou une correction de bug sans impacter directement la version principale (`main` ou `master`). On peut ensuite fusionner cette branche avec la branche principale après validation.

— **Push** : cette commande envoie les commits du dépôt local vers le dépôt distant (GitHub), pour partager les changements.

— **Pull** : cette commande permet de récupérer les modifications récentes du dépôt distant et de les fusionner avec le dépôt local.

2 Upload du projet à GitHub

Pour versionner et héberger un projet sur GitHub, il faut ouvrir un terminal dans l'emplacement du projet, puis suivre les étapes suivantes :

```
git init // Initialiser le dépôt local
git add . // Ajouter tous les fichiers/dossiers qui sont dans le même répértoire
git commit -m "Initial commit" // Faire un commit initial
git remote add origin https://github.com/nom-utilisateur/nom-depot.git // Lier le dépôt local au dépôt distant (créé sur GitHub)
git push -u origin main // Envoyer le code
```

FIGURE 9.1 – github

Remarque : Si la branche principale s'appelle `master` au lieu de `main`, adaptez la commande.

3 Problème de remote déjà existant

Il peut arriver que le projet ait déjà un dépôt distant configuré. Dans ce cas, l'ajout d'un nouveau `remote` échouera. Pour corriger cela :

```
git remote -v // Vérifier les remotes existants
git remote remove origin // Supprimer l'ancien remote
git remote add origin https://github.com/nom-utilisateur/nouveau-depot.git // Ajouter un nouveau
```

FIGURE 9.2 – Problème de remote déjà existant

4 Problème d'anciennes sessions

Lorsque plusieurs sessions Git ont été utilisées sur la même machine (ou avec différents comptes GitHub), des conflits d'authentification peuvent apparaître, notamment lors des `push` ou `pull`.

— **Vider les identifiants enregistrés :**

`Start → Credentials Manager → Windows Credentials → git:https://github.com → Re`

— **Recréer une session :**

`git push -u origin main → Login avec navigateur`

5 Modification d'un code déjà uploadé à Github

Lorsque le projet est déjà sur GitHub, les modifications doivent suivre le flux Git pour rester synchronisées entre la version locale et distante.

```
git status // Vérifier les modifications
git add . // Ajouter les fichiers modifiés
git commit -m "Explication de la modification" // Faire un commit avec un message clair
git push origin main // Envoyer les modifications vers GitHub
git push origin nom-de-branche // (Optionnel) Si vous travaillez avec des branches
```

FIGURE 9.3 – Modification d'un code déjà uploadé à Github

Remarque : si d'autres personnes ont modifié le dépôt entre-temps, un `git pull` peut être nécessaire avant de pousser à nouveau.

6 Projet en collaboration avec Github

GitHub est un excellent outil pour travailler en équipe grâce au contrôle de version et à la gestion des branches. **Principes de base pour la collaboration :**

— Chaque développeur clone le dépôt :

```
git clone https://github.com/utilisateur/projet.git
```

— On crée une nouvelle branche pour chaque fonctionnalité :

```
git checkout -b feature/nom-fonctionnalite
```

— Une fois les modifications terminées, on pousse la branche :

```
git push origin feature/nom-fonctionnalite
```

— On ouvre une **pull request** (PR) sur GitHub pour proposer l'intégration au code principal.

— Un ou plusieurs coéquipiers peuvent faire une revue de code et approuver ou demander des modifications.

— Une fois la PR validée, elle est fusionnée (merge) dans la branche `main` ou `develop`.

III Vercel et son problème

1 Déploiement simple sur Vercel

Vercel est une plateforme d'hébergement cloud spécialement conçue pour les projets frontend comme ceux réalisés avec **Next.js**. Elle offre un déploiement automatique à partir de GitHub, GitLab ou Bitbucket.

1. Aller sur le site https://vercel.com
2. Se connecter avec un compte GitHub.
3. Cliquer sur `New Project` et importer le dépôt GitHub du projet.
4. Suivre les étapes proposées par l'assistant :
   — Choisir le framework détecté automatiquement (Next.js)
   — Définir les variables d'environnement si besoin (ex : `DATABASE_URL`, `API_KEY`, etc.)
   — Lancer le déploiement
5. Le site est en ligne avec une URL du type : `https://nom-du-projet.vercel.app`

Ce système est entièrement automatisé : à chaque `git push`, Vercel re-déploie automatiquement le projet.

2 Limites et problèmes avec Vercel

Même si Vercel est parfaitement adapté aux projets **frontend statiques ou SSR**, il présente des limitations importantes pour les projets qui utilisent :

- **Express.js ou un serveur Node.js personnalisé**
- **Socket.IO (WebSockets)**
- **APIs persistantes non serverless**

Explications :

- Vercel fonctionne sur un modèle **serverless** : chaque fonction (API route) est exécutée indépendamment, à la demande.
- Il n'est pas possible d'y faire tourner un serveur Express "longue durée", ni un serveur WebSocket en écoute permanente.
- Par conséquent, les projets qui combinent `Next.js` + `Express.js` + `Socket.IO` (ou autres serveurs custom) **ne fonctionneront pas correctement sur Vercel**.

IV Hostinger

1 Problèmatique

Hostinger propose principalement des hébergements mutualisés (shared hosting) adaptés aux sites statiques ou aux applications PHP (comme WordPress). Cependant, ces hébergements ne sont **pas compatibles avec Next.js en mode serveur** (server-side rendering ou API routes).

Pourquoi cela pose problème :

- Next.js a besoin d'un environnement Node.js pour fonctionner pleinement, notamment pour les routes API, le rendu côté serveur (SSR), ou les middlewares.
- Les hébergements mutualisés chez Hostinger ne permettent pas d'exécuter un serveur Node.js personnalisé. Ils ne fournissent pas d'accès direct pour lancer des processus comme `npm run build` ou `npm start`.
- Il est donc impossible d'utiliser les fonctionnalités dynamiques de Next.js, sauf si l'on exporte le projet en site statique (avec `next export`), ce qui limite fortement ses capacités.

2 Solution VPS

L'hébergement via un VPS (Virtual Private Server) permet de déployer une application Next.js complète avec serveur Node.js personnalisé, Express, WebSockets, base de données, etc. Voici les étapes principales pour configurer un VPS (ex. Contabo) et déployer un projet :

2.1 Démarrage et accès au VPS

Après l'achat du VPS (exemple Contabo), récupérez l'adresse IP, le nom d'utilisateur (souvent `root`) et le mot de passe. Connectez-vous au VPS via PuTTY (ou un terminal SSH) : **Dans la suite, on va considérer que l'ip du vps est 207.180.194.68 et que le domaine est www..monsite.com**

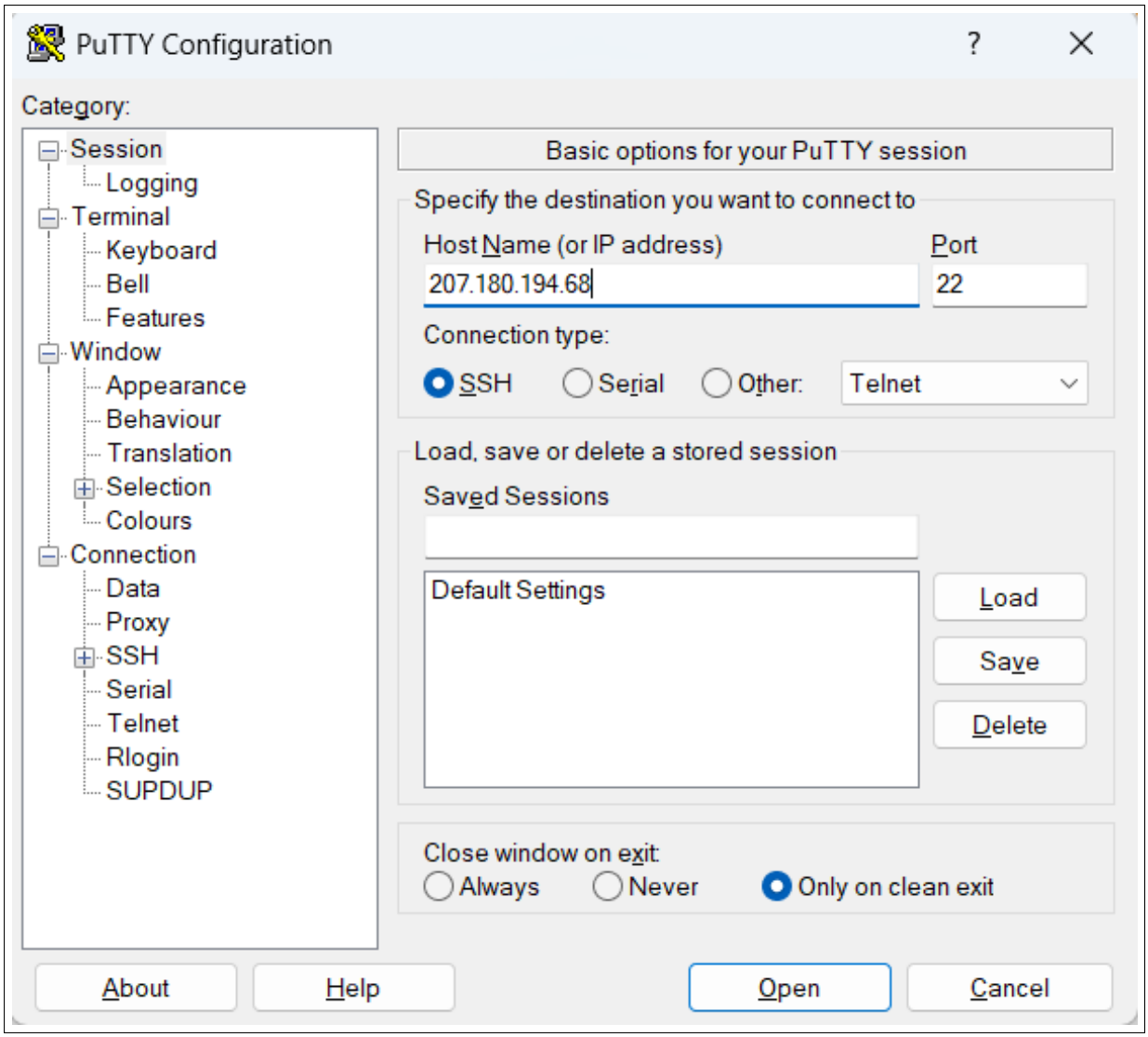


FIGURE 9.4 – Putty

2.2 Configuration du VPS

```
// 1. Installez Nginx (serveur web proxy inverse) pour faire le lien
// entre le domaine, le HTTPS,et votre application Node.js
sudo apt update
sudo apt install nginx

// 2. Configurez un fichier de site dans "/etc/nginx/sites-available/" avec
// un proxy_pass vers votre app Node.js (exemple sur le port 3000)
sudo nano /etc/nginx/sites-available/
// On met dedans :
server {
    listen 80;
    server_name monsite.com www.monsite.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}

// 3. Activez ce site
sudo ln -s /etc/nginx/sites-available/monsite /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

// 4. Sécurisation HTTPS un certificat SSL avec Certbot
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d monsite.com -d www.monsite.com

// 5. Import du projet depuis GitHub
sudo apt install git
git clone https://github.com/utilisateur/nom-depot.git
cd nom-depot
```

FIGURE 9.5 – Configuration du VPS 1

```
// 6. Navigation dans les fichiers
cd /var/www/www.monsite.com
ls
sudo nano src/pages/index.jsx

// 7. Connexion à la base de données
sudo mysql -u root -p
CREATE DATABASE myappdb CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
CREATE USER 'myuser'@'localhost' IDENTIFIED BY 'motdepassefort';
GRANT ALL PRIVILEGES ON myappdb.* TO 'myuser'@'localhost';
FLUSH PRIVILEGES;
USE myappdb;
CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(100) NOT NULL,);
EXIT;

// 8. Installation des dépendances et lancement de l'application
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs
npm install
npm run build
npm start

// 9. Utilisation de PM2 pour la gestion du processus
sudo npm install -g pm2
pm2 start npm --name "mon-app" -- start
pm2 save
pm2 startup
pm2 status         // voir les processus
pm2 restart mon-app  // redémarrer
pm2 stop mon-app     // arrêter
```

FIGURE 9.6 – Configuration du VPS 2

Cette méthode garantit un déploiement flexible et performant, avec la possibilité de gérer un serveur Node.js complet, WebSockets, API, et bases de données.

V Tâches Cron planifiées

1 Définition

Les tâches Cron sont des exécutions automatiques planifiées à des intervalles réguliers : toutes les minutes, heures, jours, etc. Elles sont utiles pour automatiser des opérations répétitives comme :

- sauvegarde automatique de la base de données,
- nettoyage de fichiers temporaires,
- relance de scripts,
- vérifications régulières.

2 Méthode sans serveur : Cron en ligne

Il existe des services gratuits pour créer des tâches planifiées à distance, sans dépendre de la configuration serveur directe. Un des plus simples est :

— **cron-job.org** : https://cron-job.org

3 Étapes sur cron-job.org

1. Créer un compte gratuit sur https://cron-job.org.
2. Cliquer sur `Create cronjob`.
3. Renseigner l'URL de votre API ou script à exécuter régulièrement :

   ```
   https://monsite.com/api/cron/cleanup
   ```

4. Choisir la fréquence d'exécution (ex. tous les jours à 2h du matin).
5. Activer les notifications si souhaité.
6. Enregistrer.

VI Amélioration de compatibilité

1 Transformation en desktop-app avec Electron.js

1.1 Qu'est-ce qu'Electron.js ?

Electron.js est un framework permettant de créer des applications desktop multiplateformes (Windows, macOS, Linux) à partir d'un site web. Il utilise Chromium pour l'affichage et Node.js pour le backend local.

1.2 Étapes de transformation

1. Installer Electron :

   ```
   npm install electron --save-dev
   ```

2. Créer un fichier `main.js` :

```
1  const { app, BrowserWindow } = require('electron');
2  function createWindow() {
3    const win = new BrowserWindow({ width: 1024, height: 768 });
4    win.loadURL('http://localhost:3000'); // ou un build local
5  }
6  app.whenReady().then(createWindow);
7
```

FIGURE 9.7 – Electronjs

3. Ajouter un script dans `package.json` :

```
"scripts": {
  "start": "electron main.js"
}
```

4. Lancer l'application :

```
npm start
```

2 Transformation en mobile-app avec Webintoapp

2.1 Qu'est-ce que Webintoapp ?

https://webintoapp.com est une plateforme gratuite ou payante qui permet de convertir un site web responsive en application mobile (Android principalement), sous forme de WebView.

2.2 Étapes à suivre

1. Aller sur https://webintoapp.com.
2. Coller l'URL de votre site web (ex. https ://www.monsite.com).
3. Personnaliser :
   — Icône de l'app,
   — Splash screen,
   — Permissions,
   — Couleurs et titre.
4. Générer l'APK.
5. Télécharger l'APK et l'installer sur mobile ou publier sur le Play Store.

VII Docker

Chapitre 10 : CGI et animations haut niveau avec Three.js

Chapitre 10

CGI et animations haut niveau